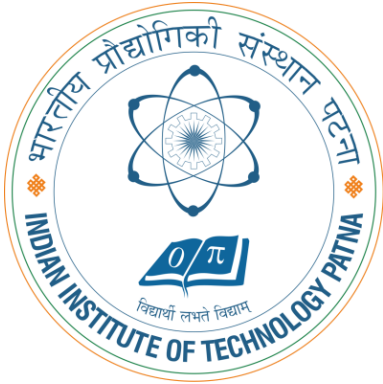


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

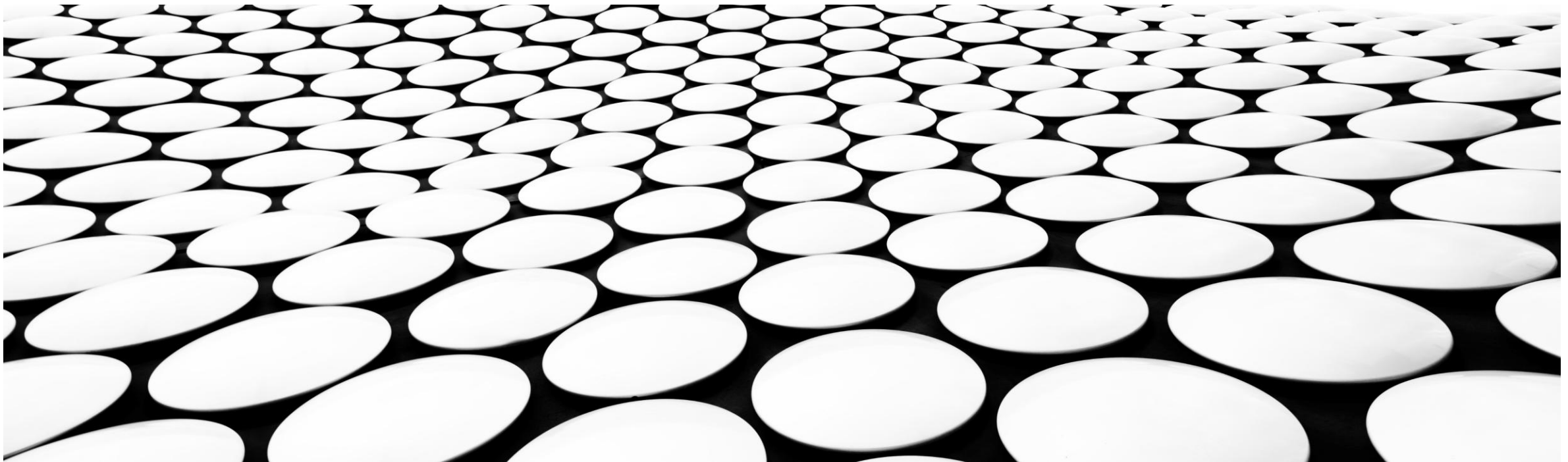


TOPIC-11: CONTROL UNIT DESIGN II

DR. ARIJIT ROY

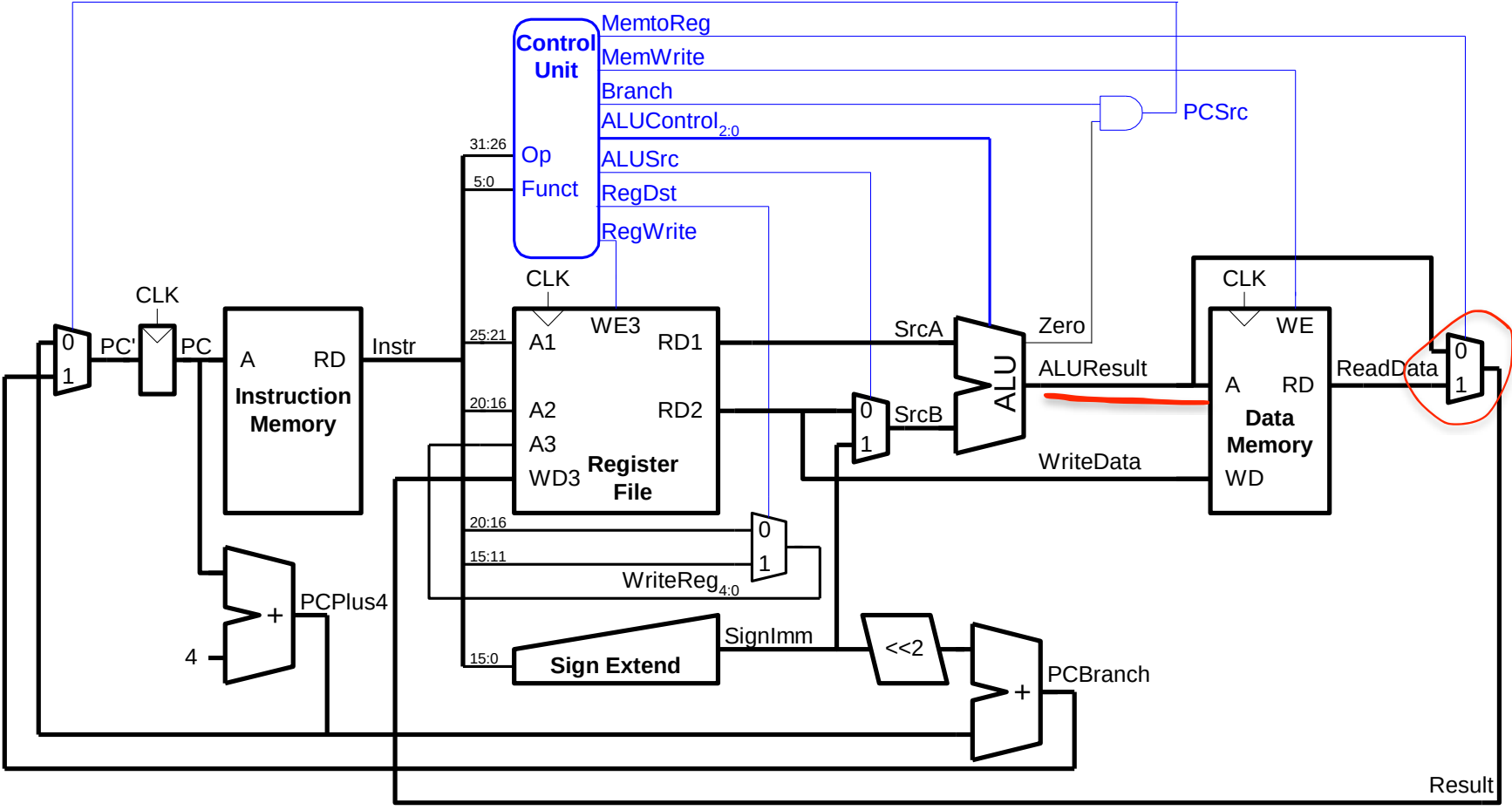
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA

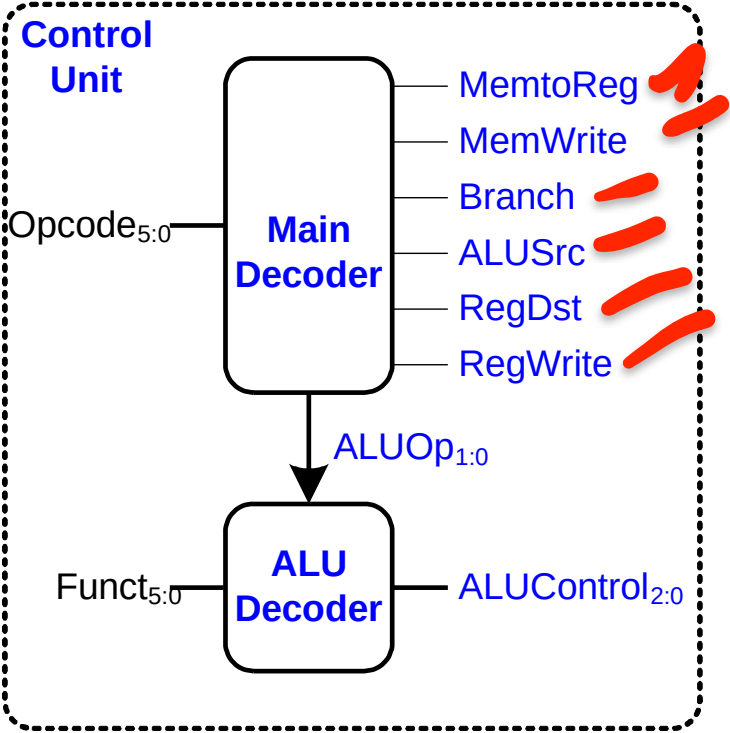


RECAP: CONTROL SIGNALS

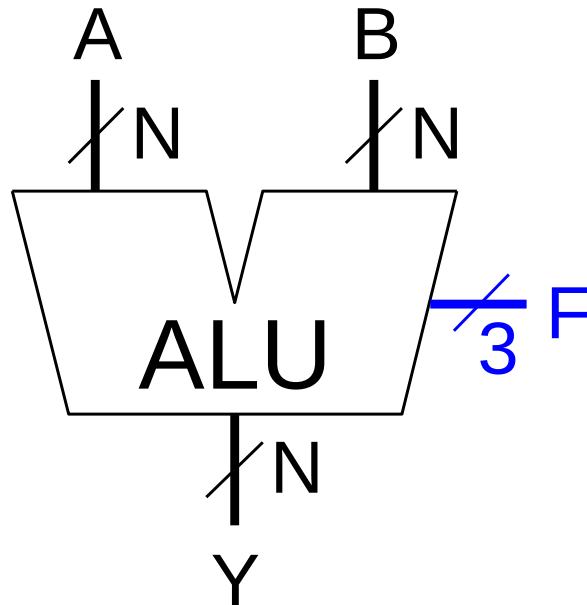
COMPLETE SINGLE-CYCLE PROCESSOR



CONTROL UNIT



REVIEW: ALU

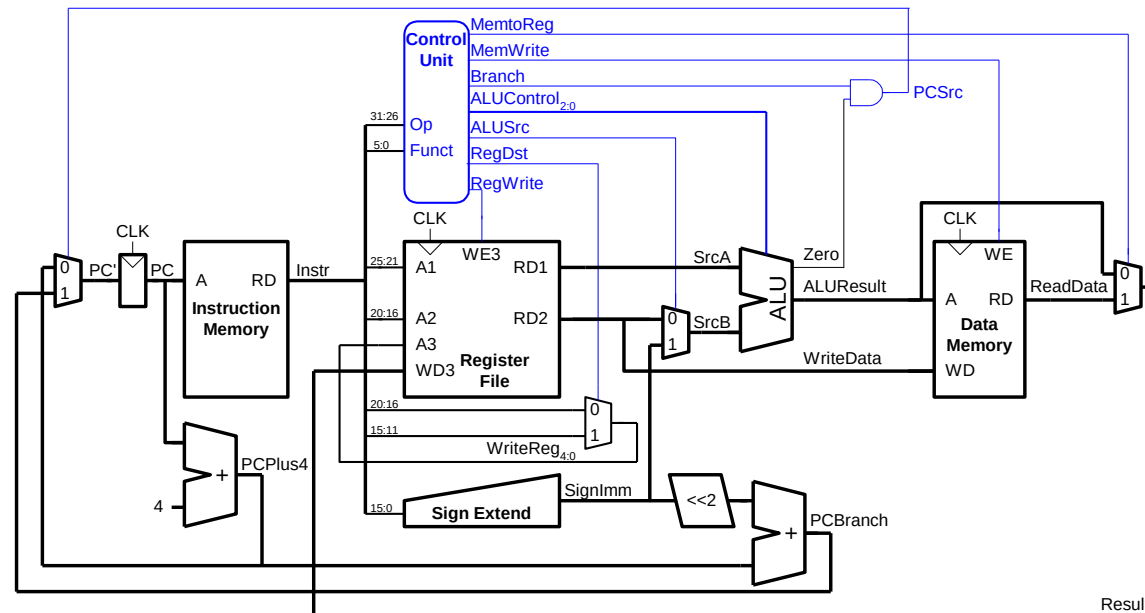


$F_{2:0}$	Function
000	$A \& B$
001	$A B$
010	$A + B$
011	not used
100	$A \& \sim B$
101	$A \sim B$
110	$A - B$
111	SLT

CONTROL UNIT: MAIN DECODER



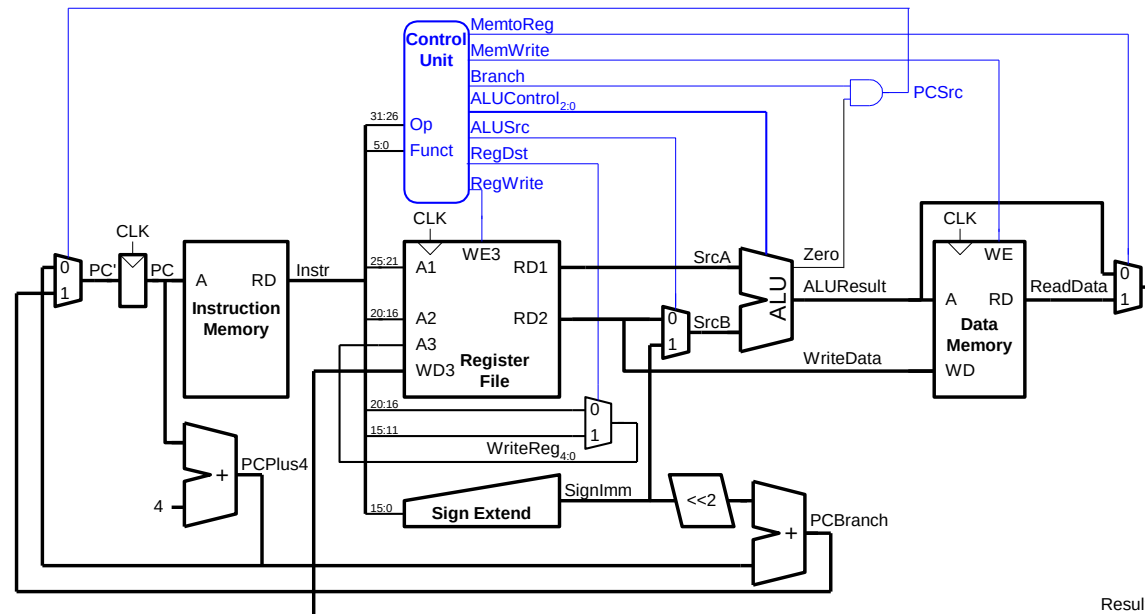
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000							
lw	100011							
sw	101011							
beq	000100							



CONTROL UNIT: MAIN DECODER



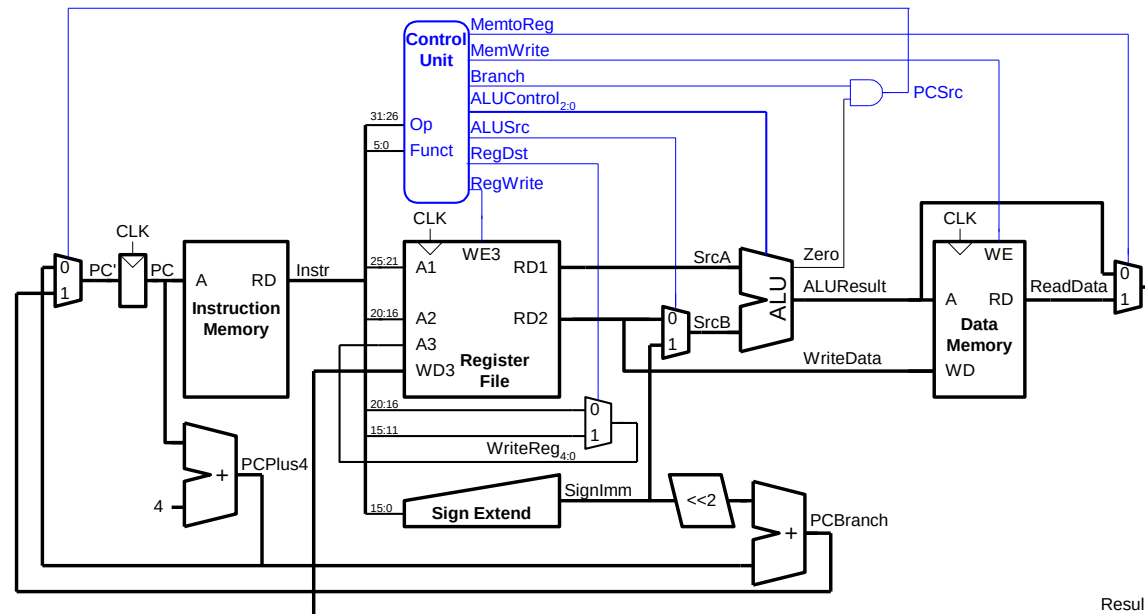
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011							
sw	101011							
beq	000100							



CONTROL UNIT: MAIN DECODER



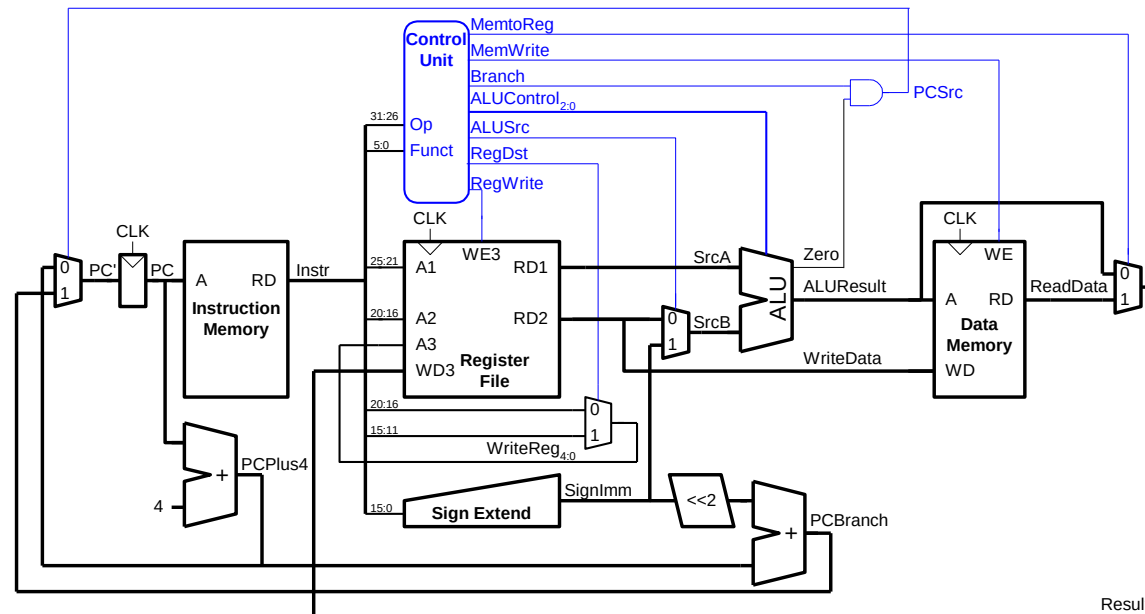
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011							
beq	000100							



CONTROL UNIT: MAIN DECODER



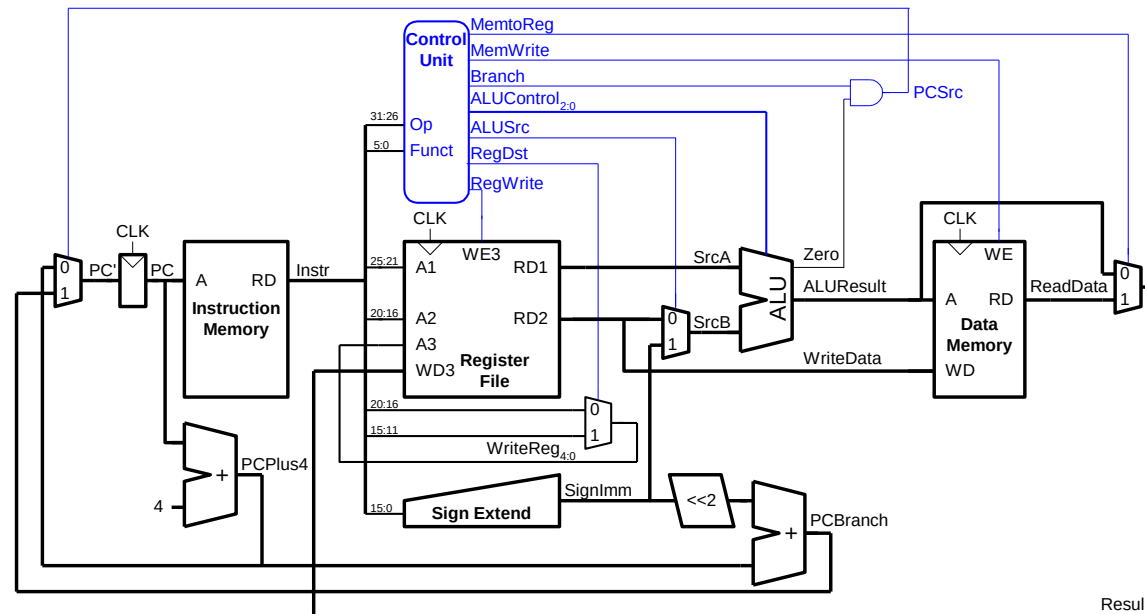
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100							



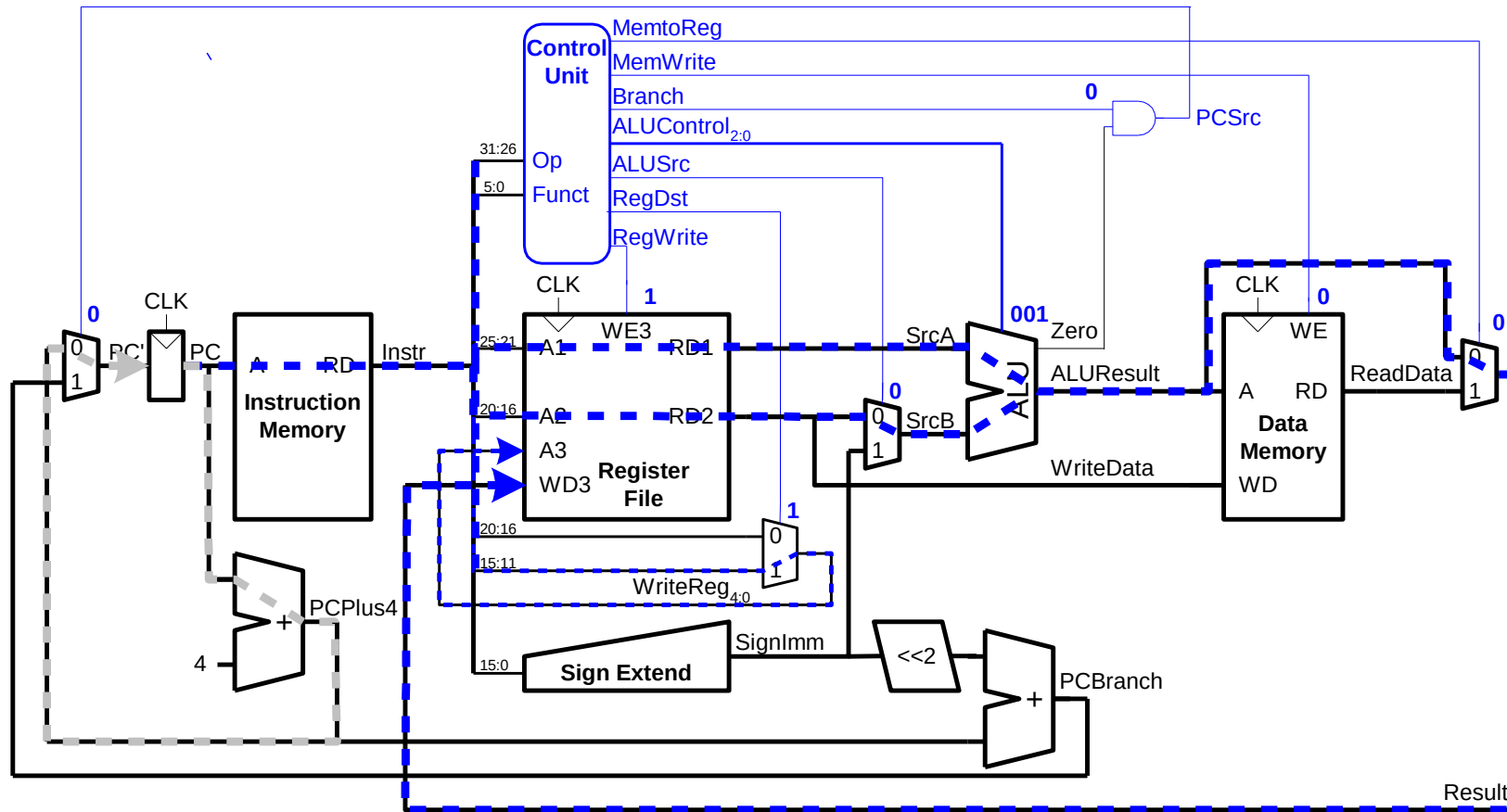
CONTROL UNIT: MAIN DECODER



Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01



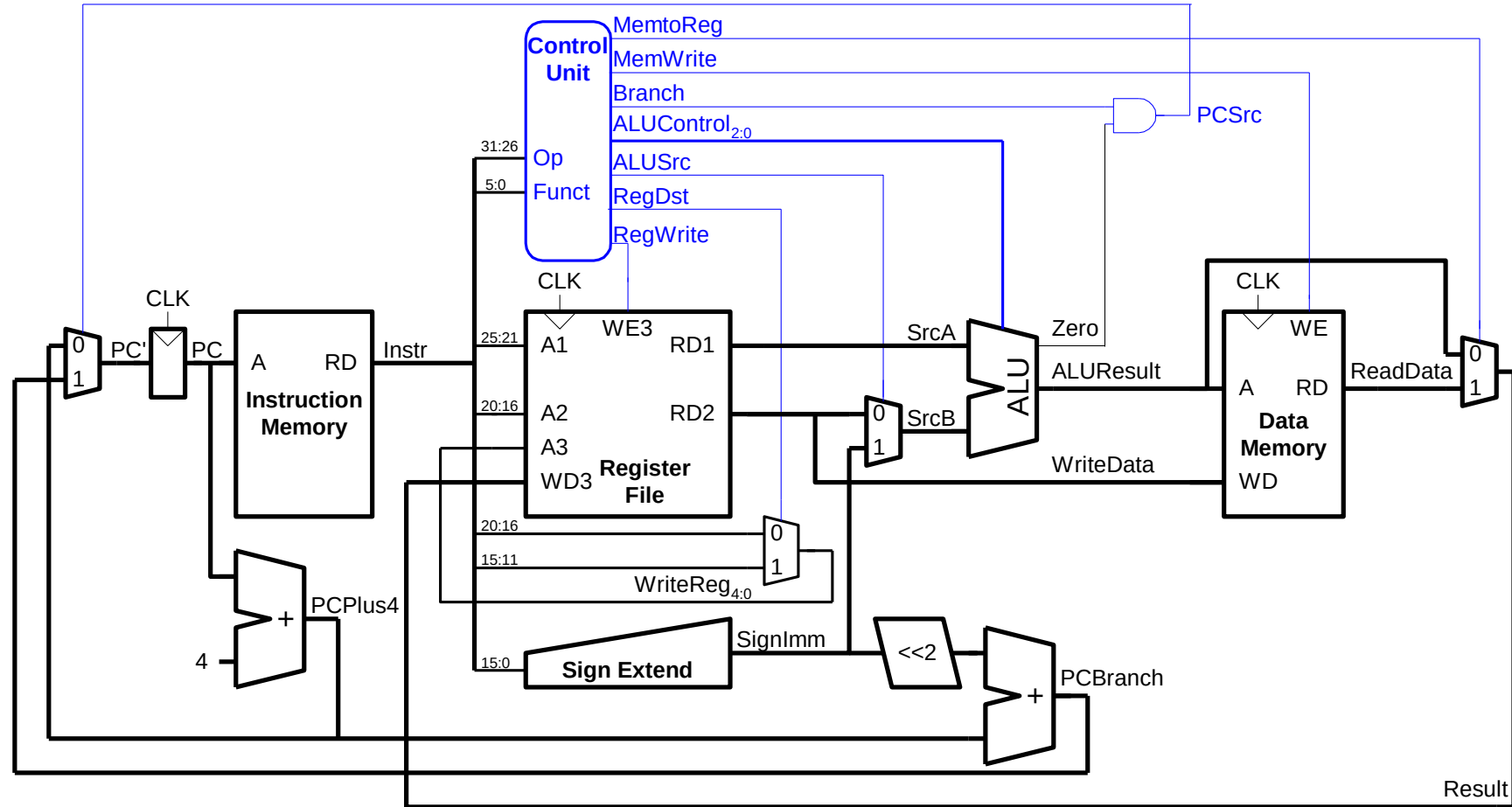
SINGLE-CYCLE DATAPATH EXAMPLE: OR



EXTENDED FUNCTIONALITY: ADDI



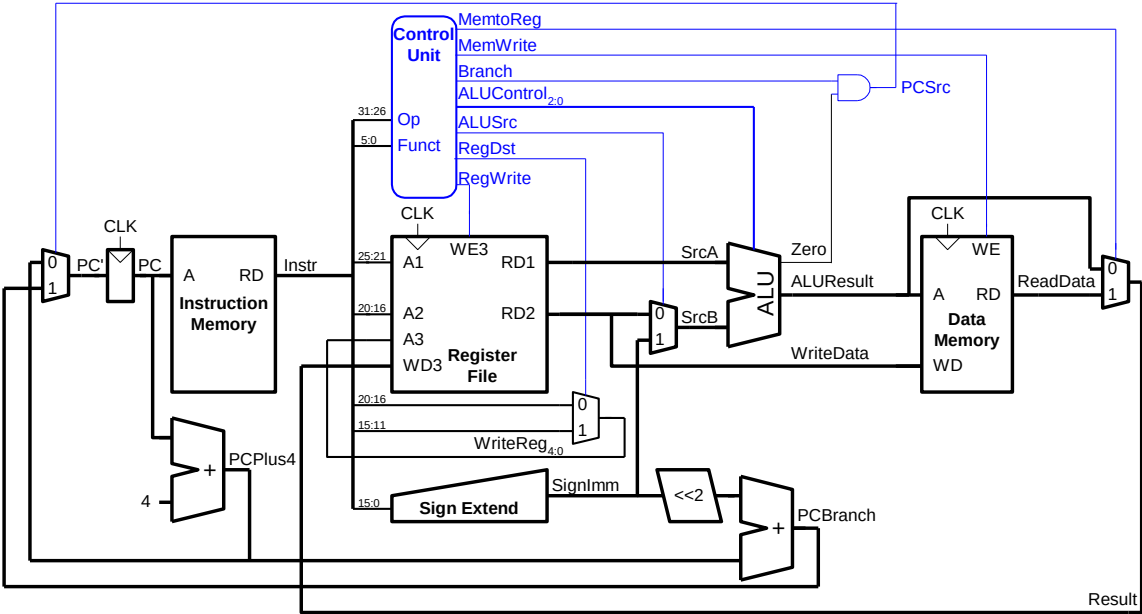
- No change to datapath



CONTROL UNIT: ADDI



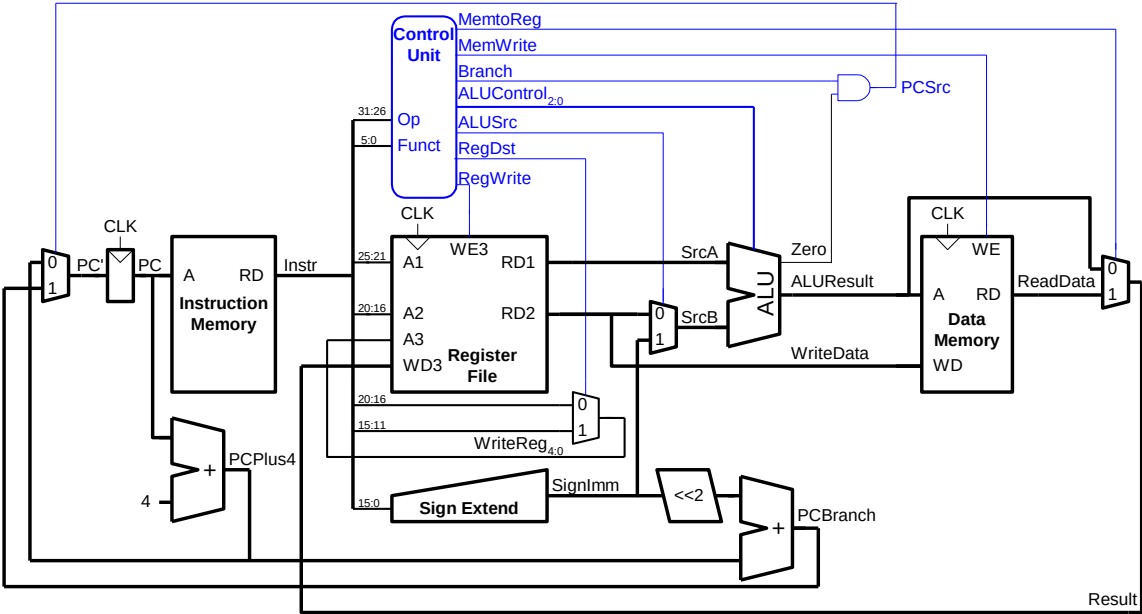
Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000							



CONTROL UNIT: ADDI

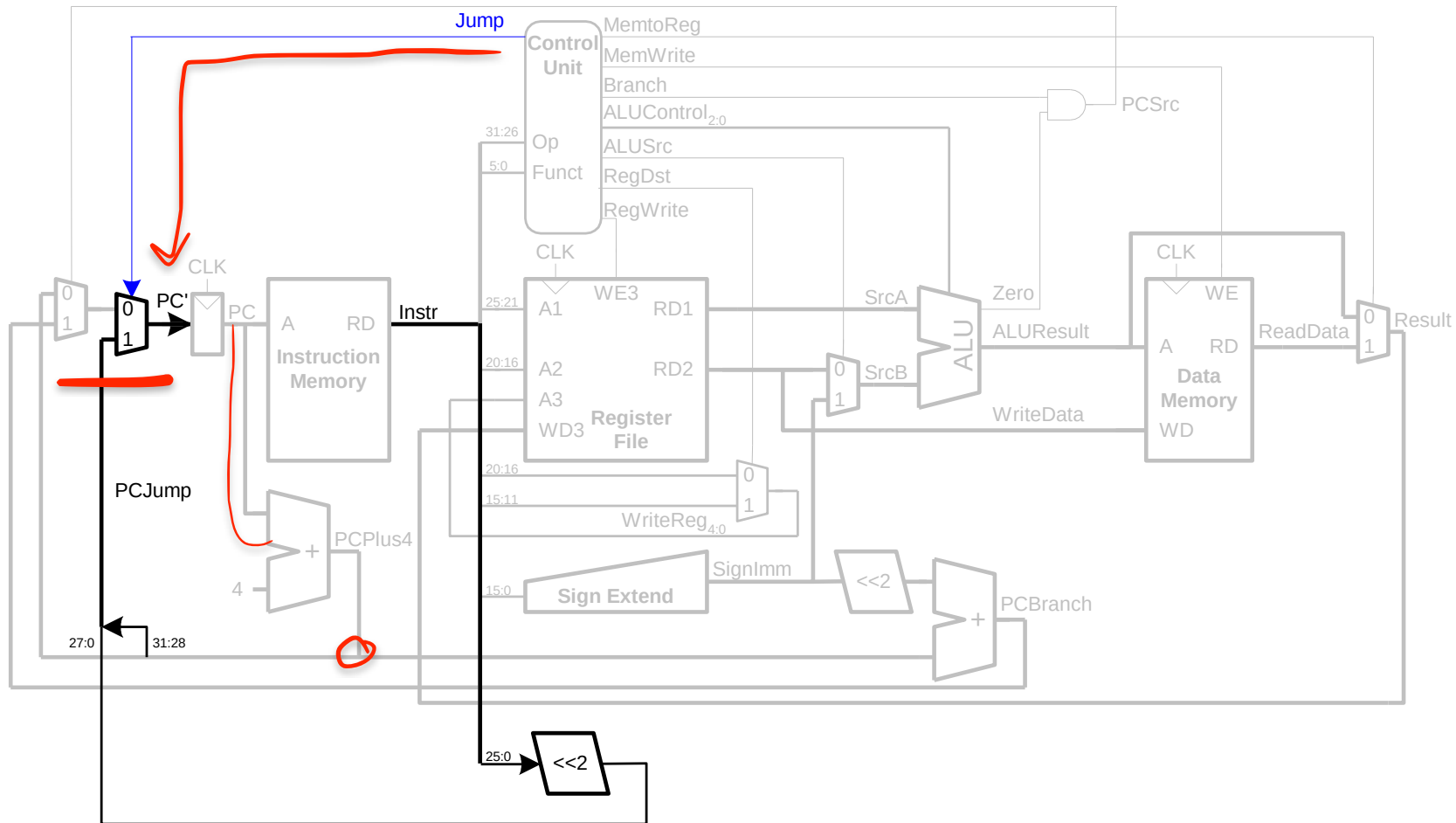


Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	001000	1	0	1	0	0	0	00



JUMP INSTRUCTION

EXTENDED FUNCTIONALITY: J



CONTROL UNIT: MAIN DECODER



Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
j	000100								

CONTROL UNIT: MAIN DECODER



Instruction	Op _{5:0}	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	ALUOp _{1:0}	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
j	000100	0	X	X	X	0	X	XX	1

REVIEW: PROCESSOR PERFORMANCE



Program Execution Time

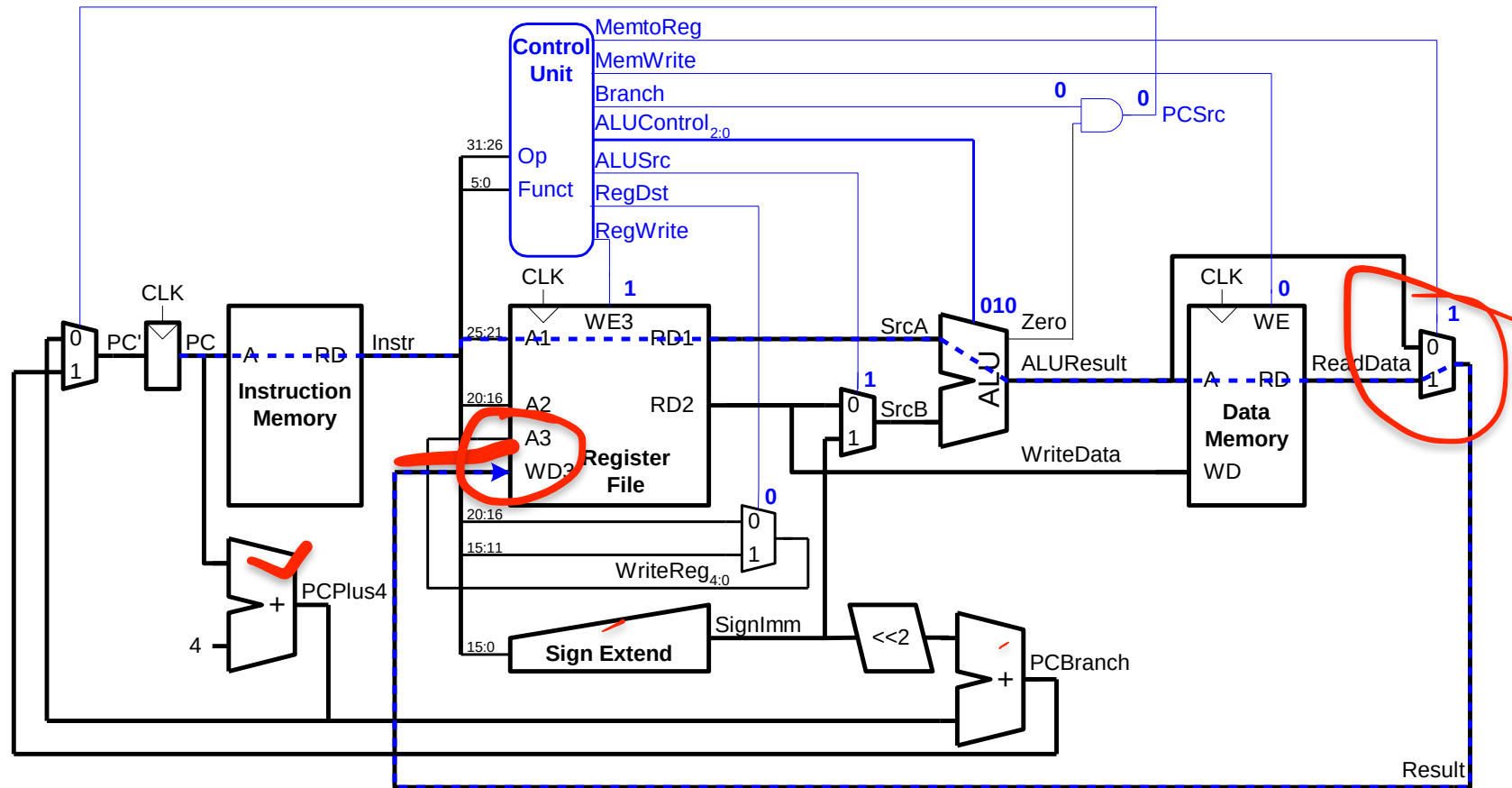
$$= (\# \text{ instructions})(\text{cycles/instruction})(\text{seconds/cycle})$$

$$= \# \text{ instructions} \times \text{CPI} \times \text{TC}$$

SINGLE-CYCLE PERFORMANCE



- T_C is limited by the critical path (1w)



SINGLE-CYCLE PERFORMANCE

➤ Single-cycle critical path:

$$T_c = t_{pcq_PC} + t_{mem} + \max(t_{RFread}, t_{ext} + t_{mux}) + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup}$$

Program counter

Memory access

Maximum of read from the register files, sign extension, and mux

ALU execution

Data memory

mux

Setting up the register file

➤ In most implementations, limiting paths are:

➤ memory, ALU, register file.

$$T_c = t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup}$$

SINGLE-CYCLE PERFORMANCE EXAMPLE



Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$T_c =$$

SINGLE-CYCLE PERFORMANCE EXAMPLE



Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30 ✓
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$\begin{aligned}T_c &= t_{pcq_PC} + 2t_{mem} + t_{RFread} + t_{mux} + t_{ALU} + t_{RFsetup} \\&= [30 + 2(250) + 150 + 25 + 200 + 20] \text{ ps} \\&= 925 \text{ ps}\end{aligned}$$

SINGLE-CYCLE PERFORMANCE EXAMPLE



- For a program with 100 billion instructions executing on a single-cycle MIPS processor,

Execution Time =

SINGLE-CYCLE PERFORMANCE EXAMPLE



- For a program with 100 billion instructions executing on a single-cycle MIPS processor,

$$\begin{aligned}\text{Execution Time} &= \# \text{ instructions} \times \text{CPI} \times T_C \\ &= (100 \times 10^9)(1)(925 \times 10^{-12} \text{ s}) \\ &= 92.5 \text{ seconds}\end{aligned}$$

MULTICYCLE MIPS PROCESSOR

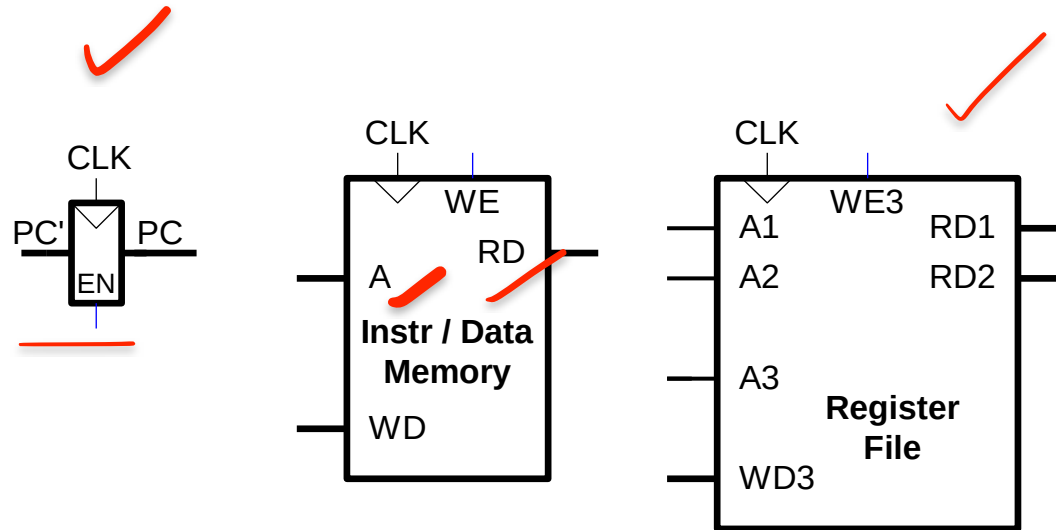
MULTICYCLE MIPS PROCESSOR



- Single-cycle microarchitecture:
 - +simple ✓
 - cycle time limited by longest instruction (lw)
 - two adders/ALUs and two memories
- Multicycle microarchitecture:
 - + higher clock speed ✓
 - + simpler instructions run faster ✓
 - + reuse expensive hardware on multiple cycles ✓
 - - sequencing overhead paid many times
- Same design steps: datapath & control

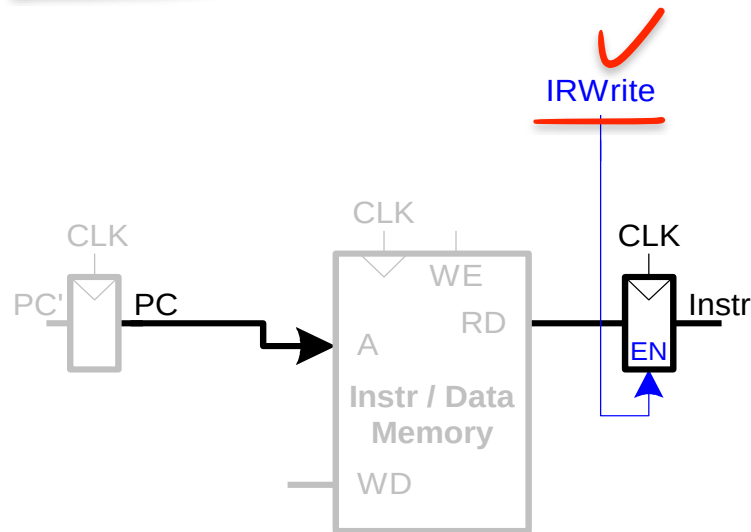
MULTICYCLE STATE ELEMENTS

- Replace Instruction and Data memories with a **single unified memory**
 - More realistic

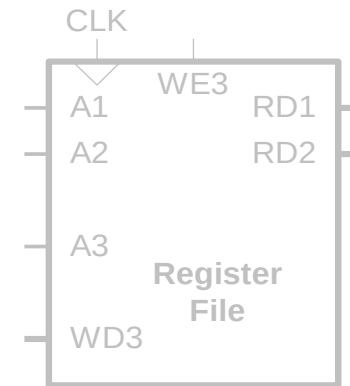


MULTICYCLE DATAPATH: INSTRUCTION FETCH

- First consider executing 1_w
- **STEP 1:** Fetch instruction



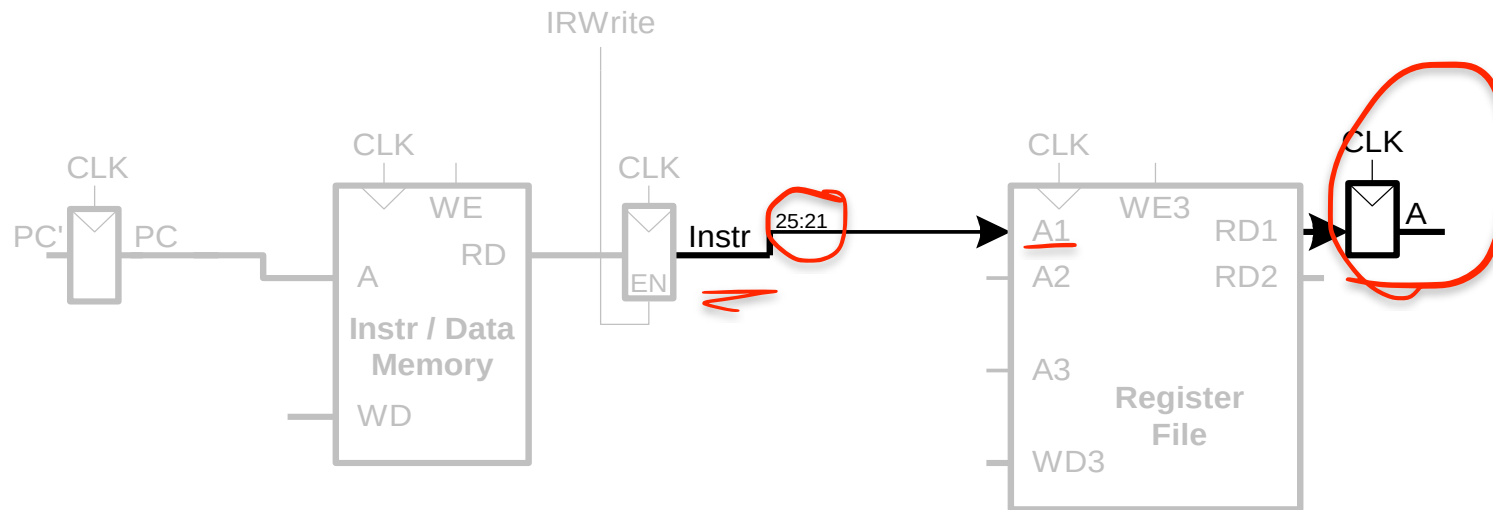
Instruction Register (IR): IR remembers the instruction even if we are in a different step



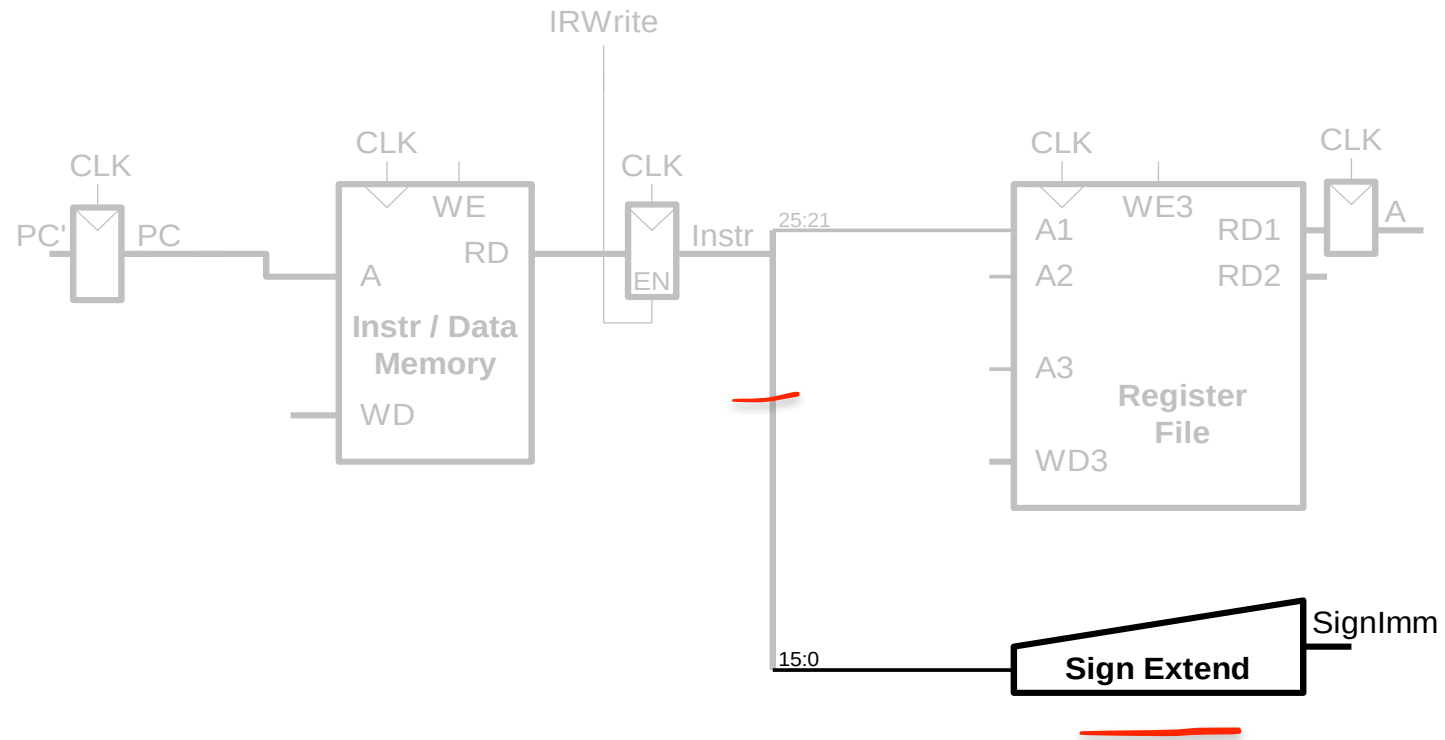
MULTICYCLE DATAPATH: lw REGISTER READ

- First consider executing lw
- **STEP 2:** Read source operand from RD

Store the content of the register in A



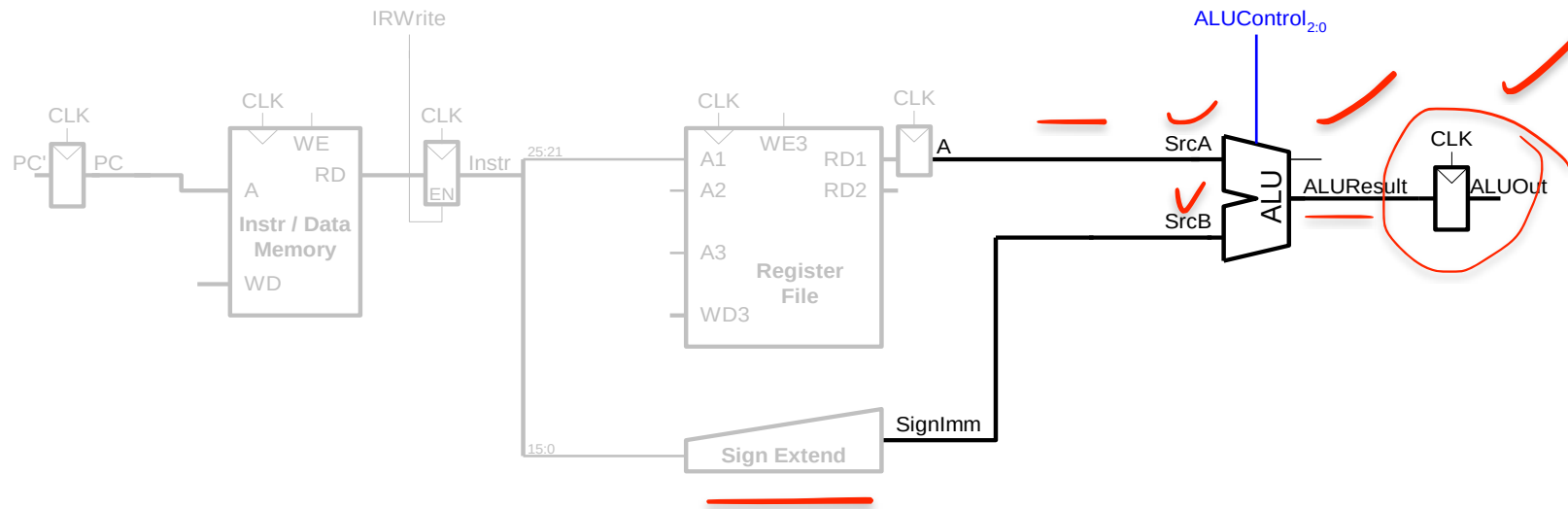
MULTICYCLE DATAPATH: LW IMMEDIATE



MULTICYCLE DATAPATH: lw ADDRESS

- First consider executing lw
- **STEP 3:** Compute the memory address

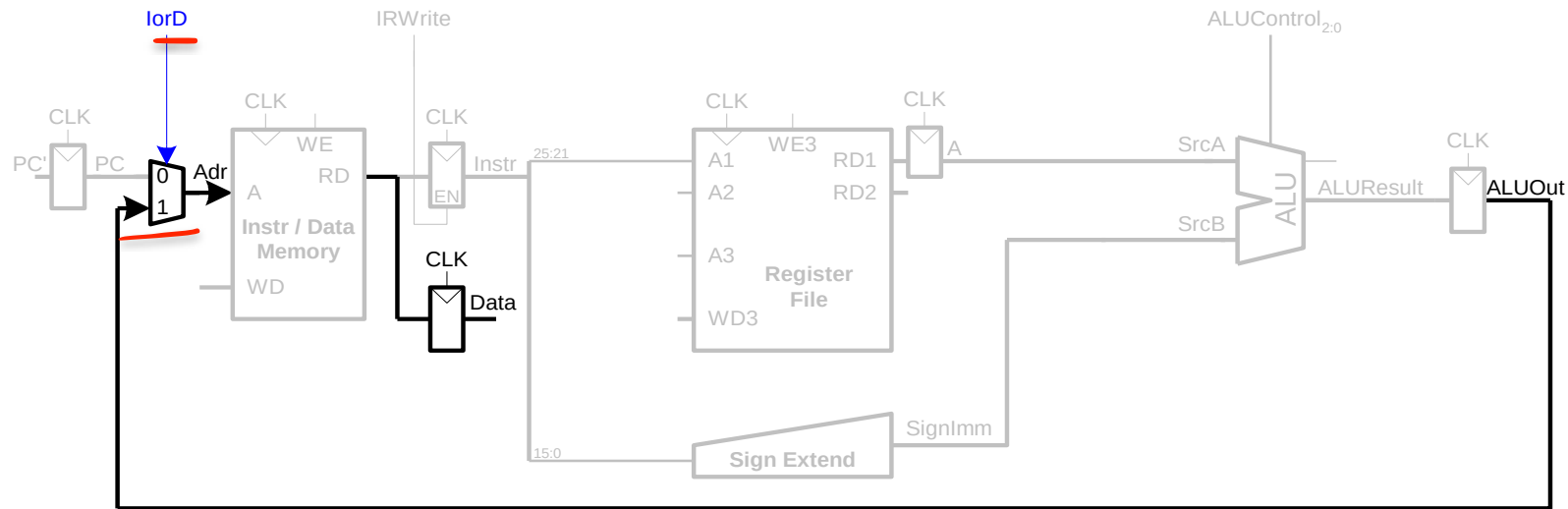
After computation, store
it in another register



MULTICYCLE DATAPATH: l_w MEMORY READ

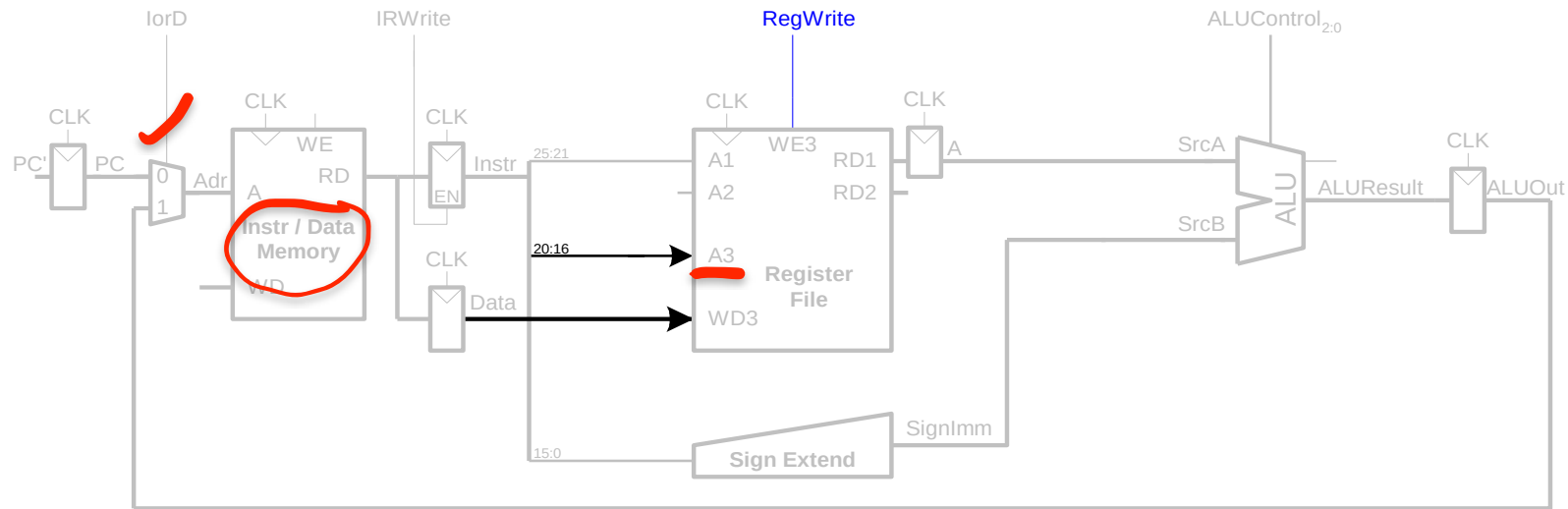
- First consider executing l_w
- **STEP 4:** Read data from memory

Multiplexer is used to determine the source, after reading, store the data in to data register

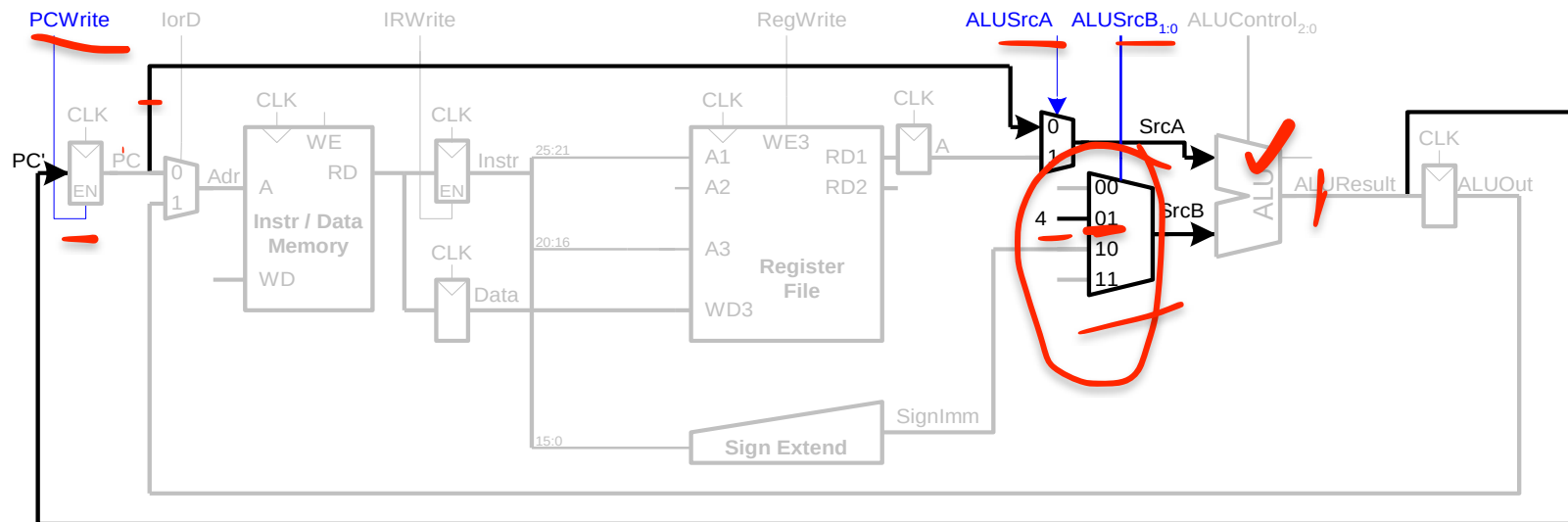


MULTICYCLE DATAPATH: lw WRITE REGISTER

- First consider executing lw
- **STEP 5:** Write back to the register file

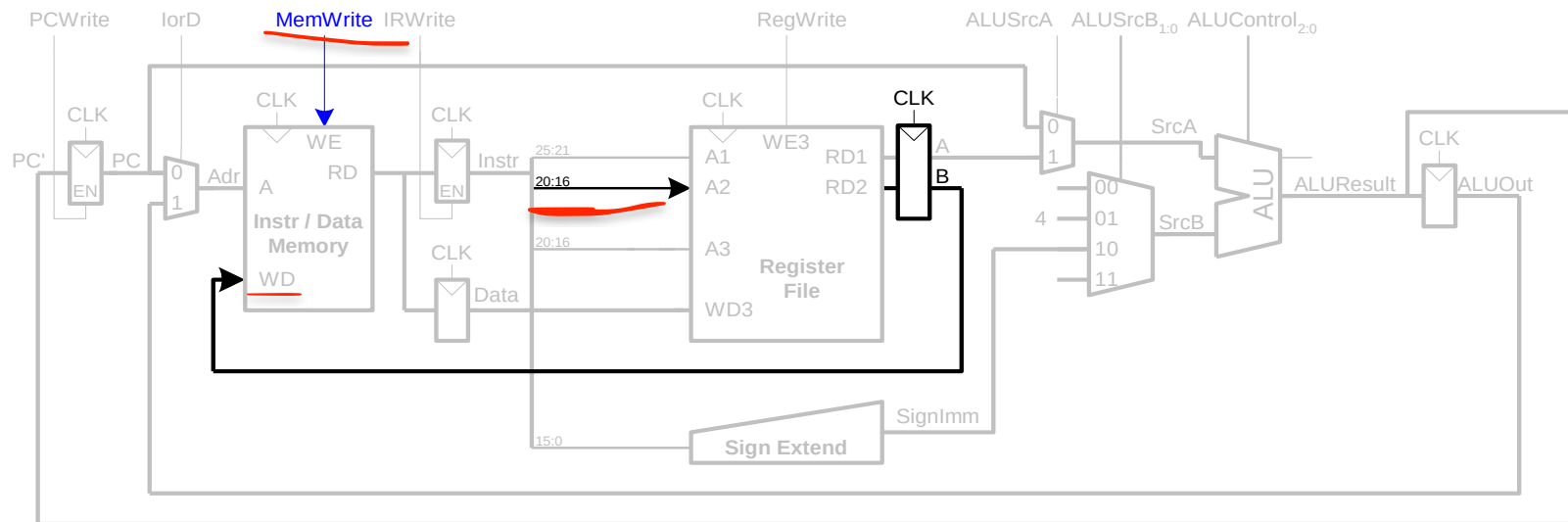


MULTICYCLE DATAPATH: INCREMENT PC



MULTICYCLE DATAPATH: SW

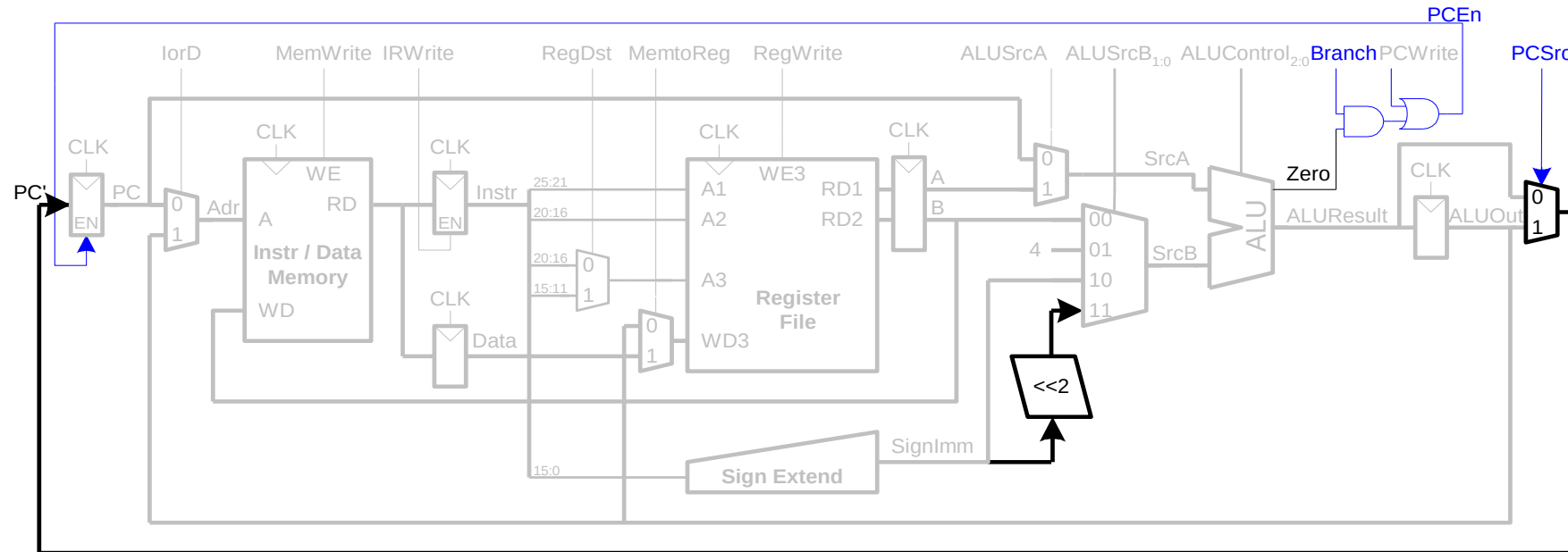
- Write data in rt to memory



MULTICYCLE DATAPATH: BEQ

- Determine whether values in rs and rt are equal
- Calculate branch target address:

$$\text{BTA} = (\text{sign-extended immediate} \ll 2) + (\text{PC} + 4)$$

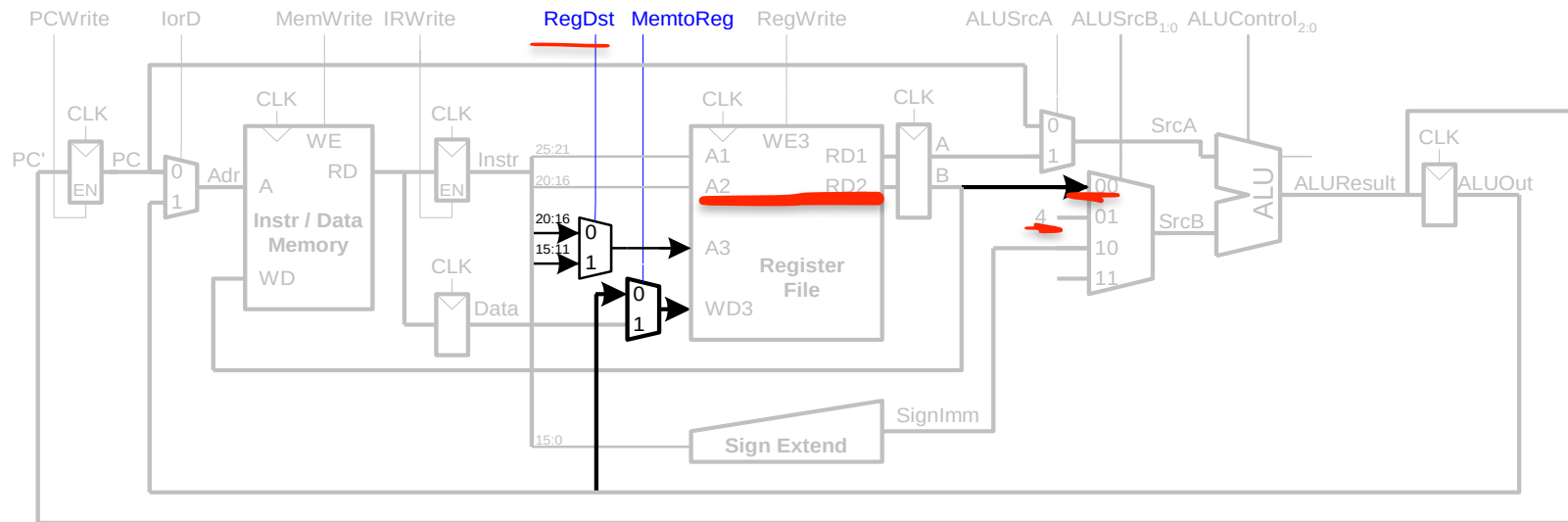


BRANCH TARGET ADDRESS DETAILS

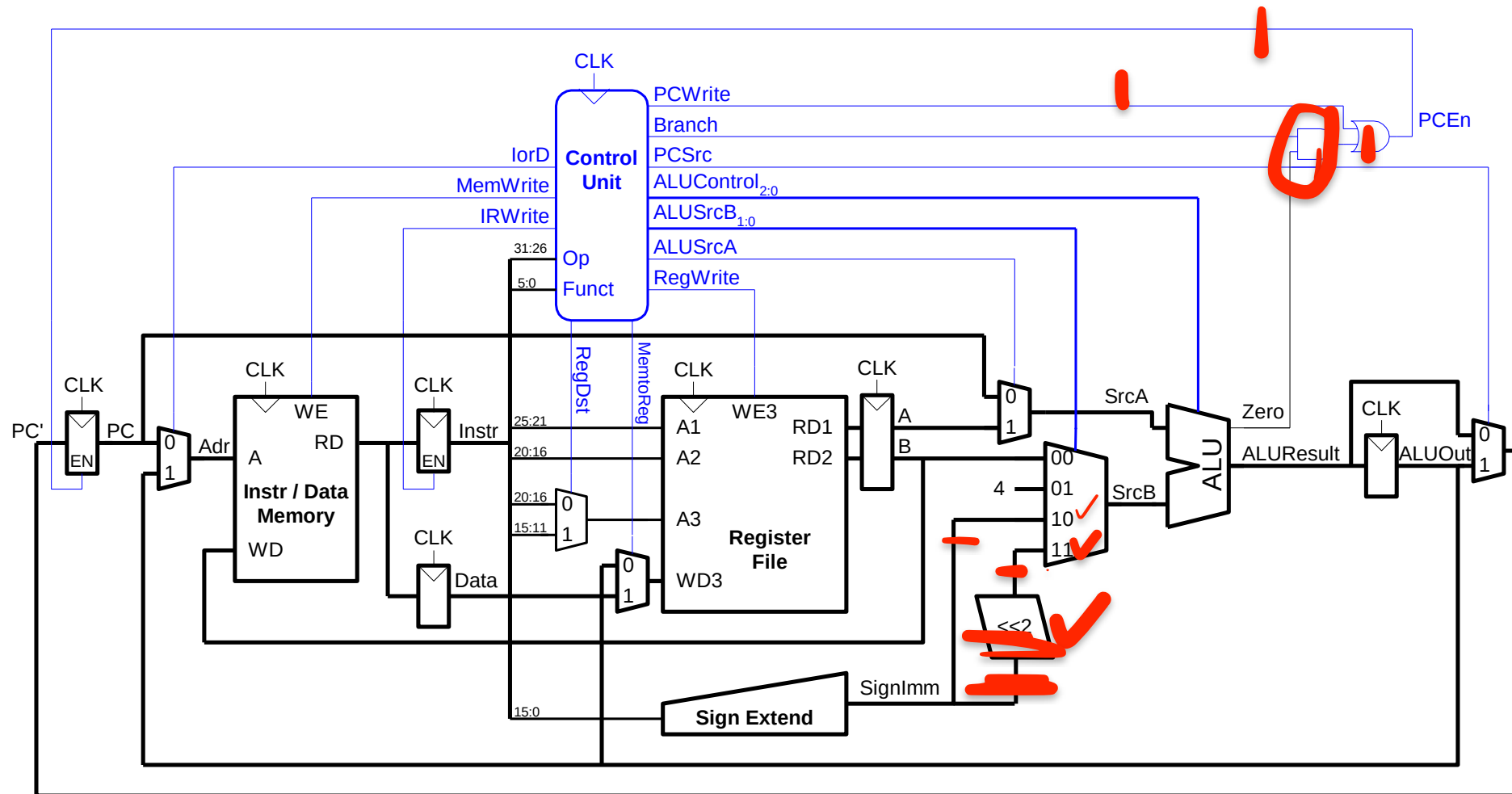
- $BTA = (\text{sign-extended immediate} \ll 2) + (PC+4)$
- **Sign-Extended Immediate:** This refers to the immediate value, a constant embedded within the instruction, that has been extended to match the processor's word size (e.g., 32 bits) by replicating its sign bit. Sign extension preserves the immediate value's positive or negative nature during operations.
- **Shift Left by 2 ($\ll 2$):** The immediate value is shifted left by 2 bits, effectively multiplying it by 4. This shift is necessary because MIPS and similar architectures use word-aligned memory, where each memory address corresponds to a 4-byte word. The immediate value usually indicates the number of instructions to jump, and multiplying by 4 converts this to the actual byte offset.
- **Program Counter + 4 ($PC + 4$):** The $PC + 4$ component reflects that the program counter (PC) typically points to the current instruction's address. Adding 4 to the PC advances it to the next instruction (since each instruction occupies 4 bytes). This value serves as the base address to which the branch offset (calculated using the shifted immediate value) is applied.

MULTICYCLE DATAPATH: R-TYPE INSTRUCTIONS

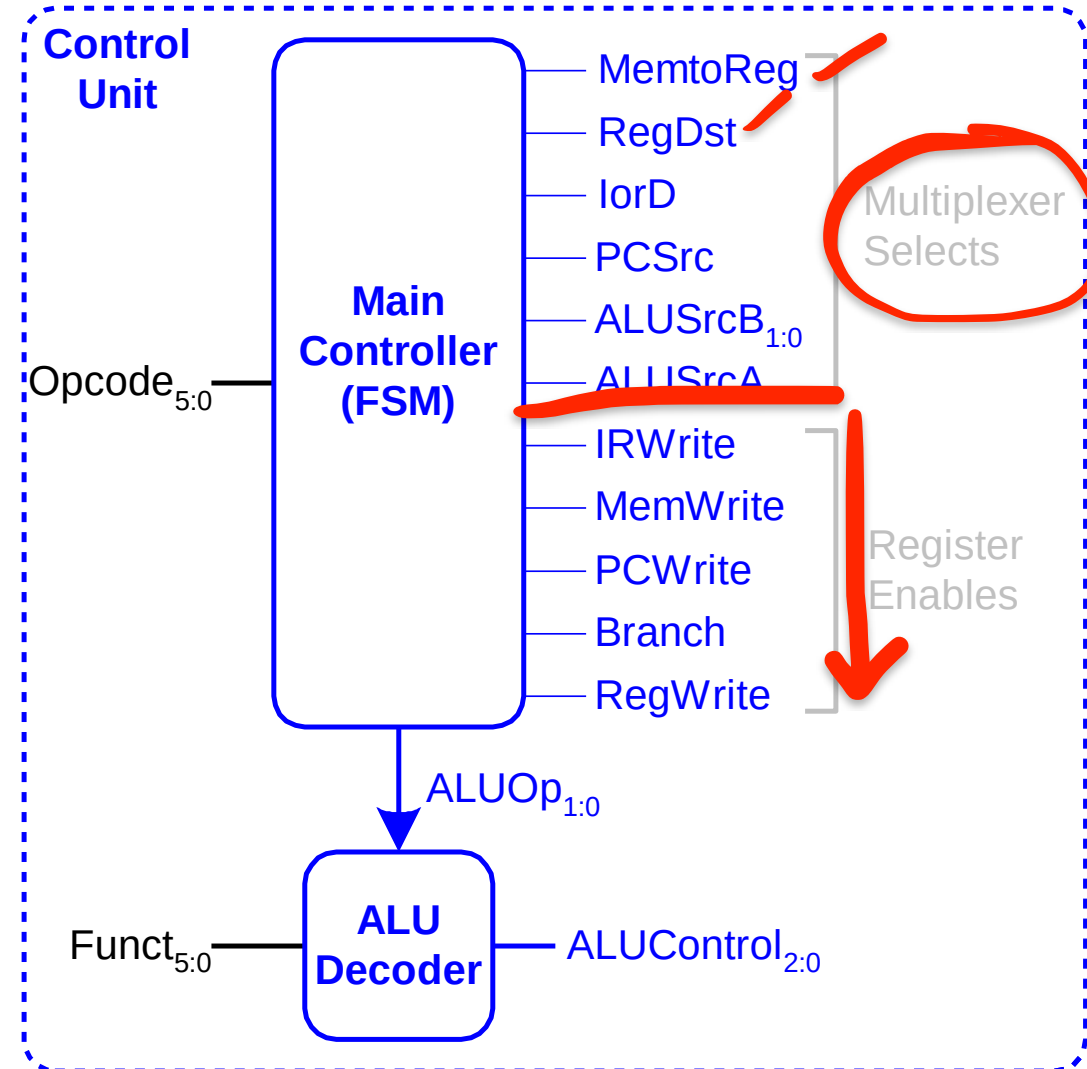
- Read from rs and rt
- Write $ALUResult$ to register file
- Write to rd (instead of rt)



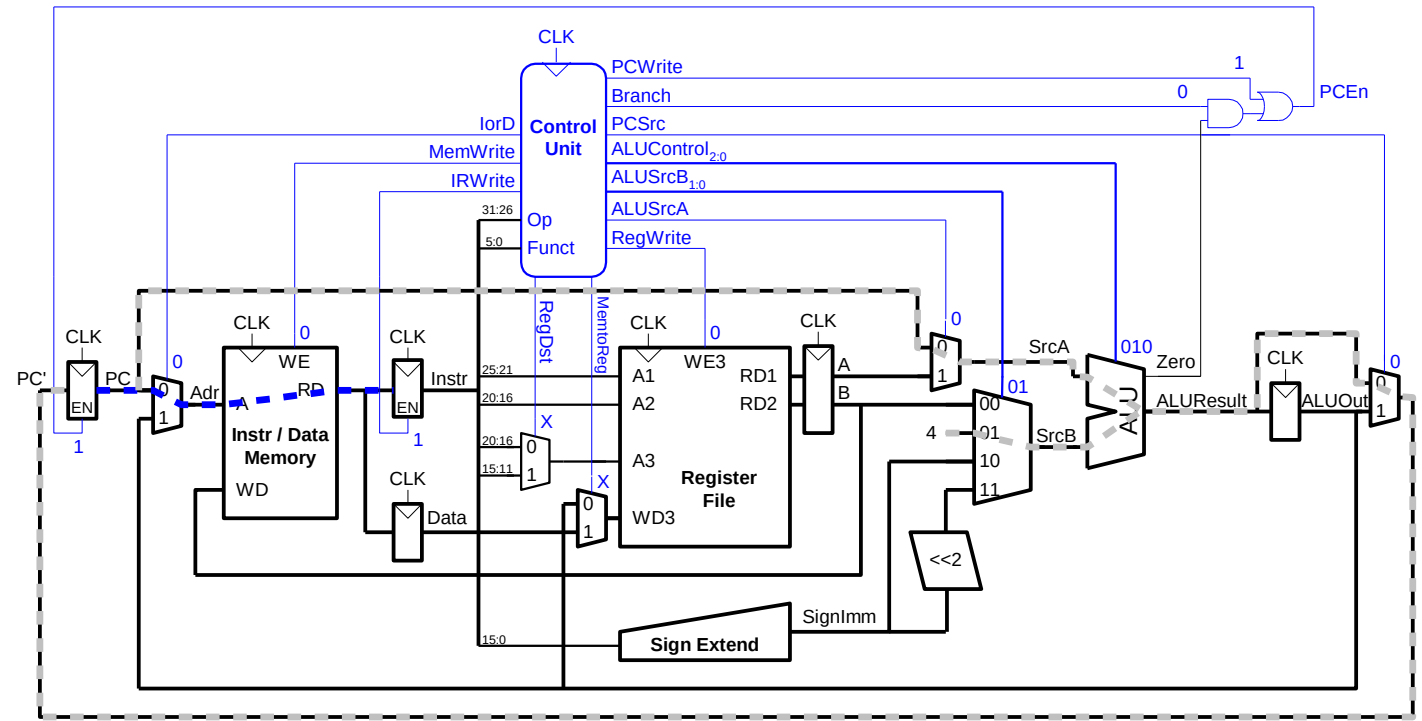
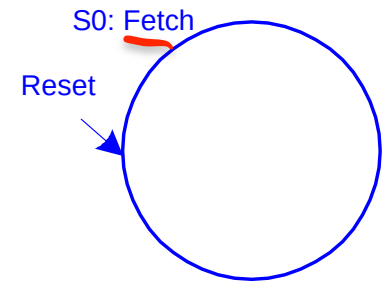
COMPLETE MULTICYCLE PROCESSOR



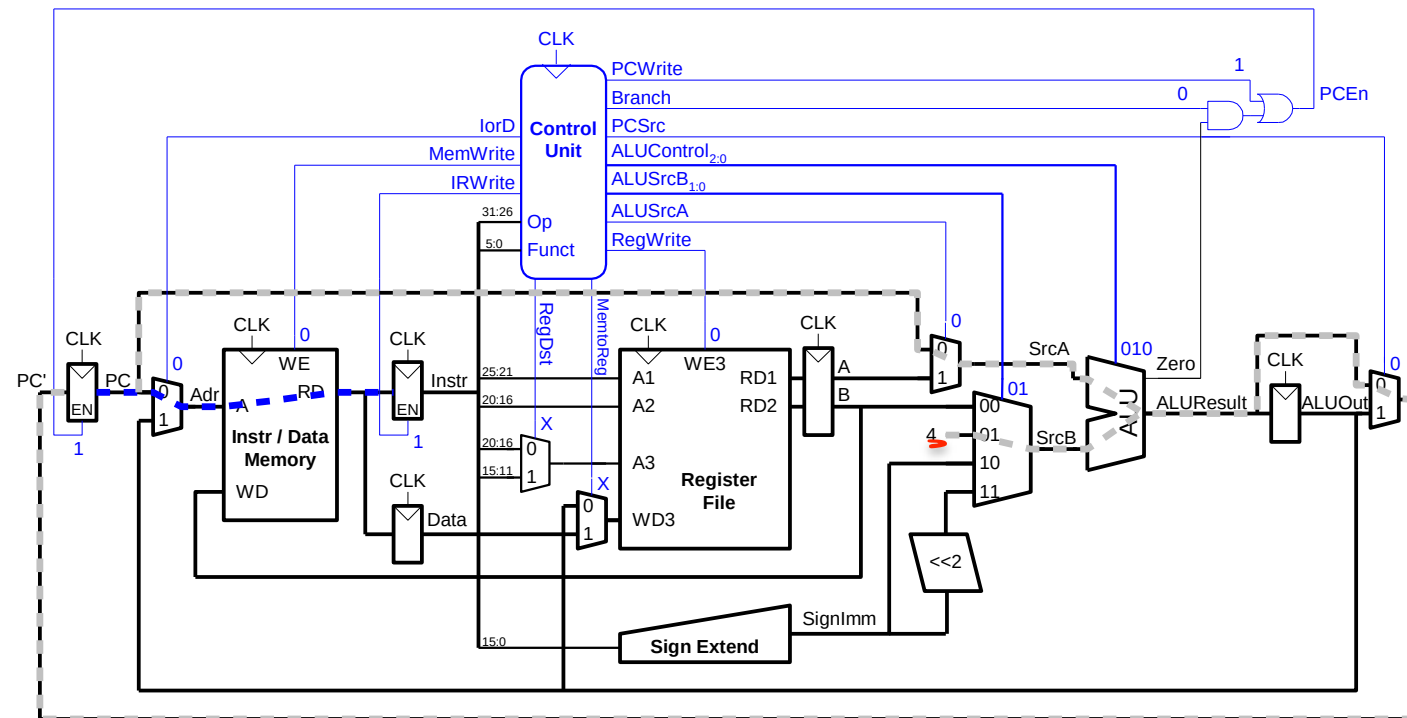
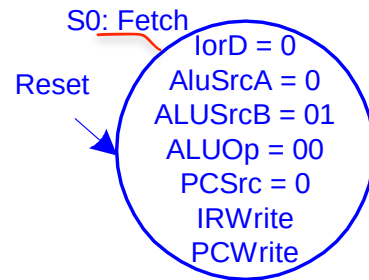
CONTROL UNIT



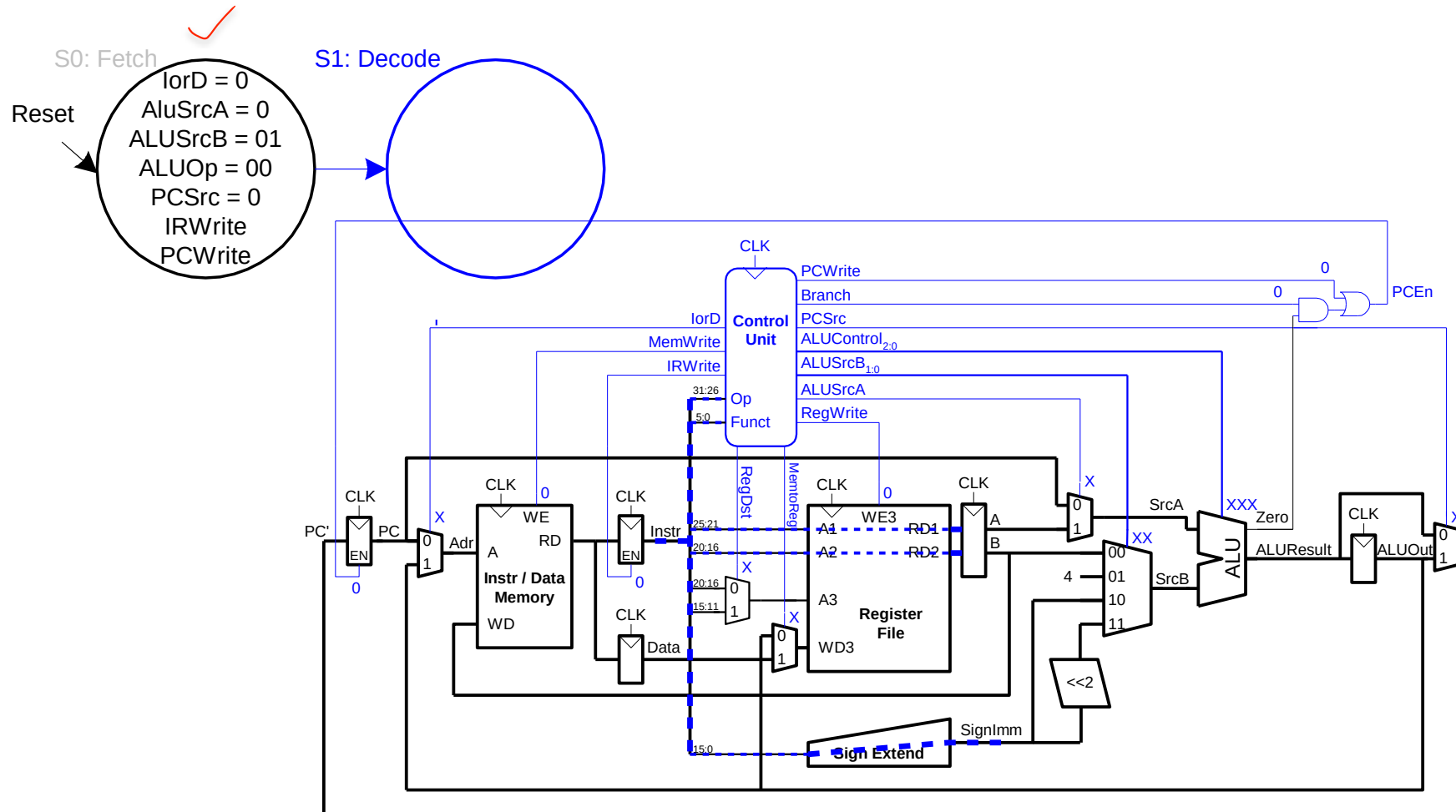
MAIN CONTROLLER FSM: FETCH



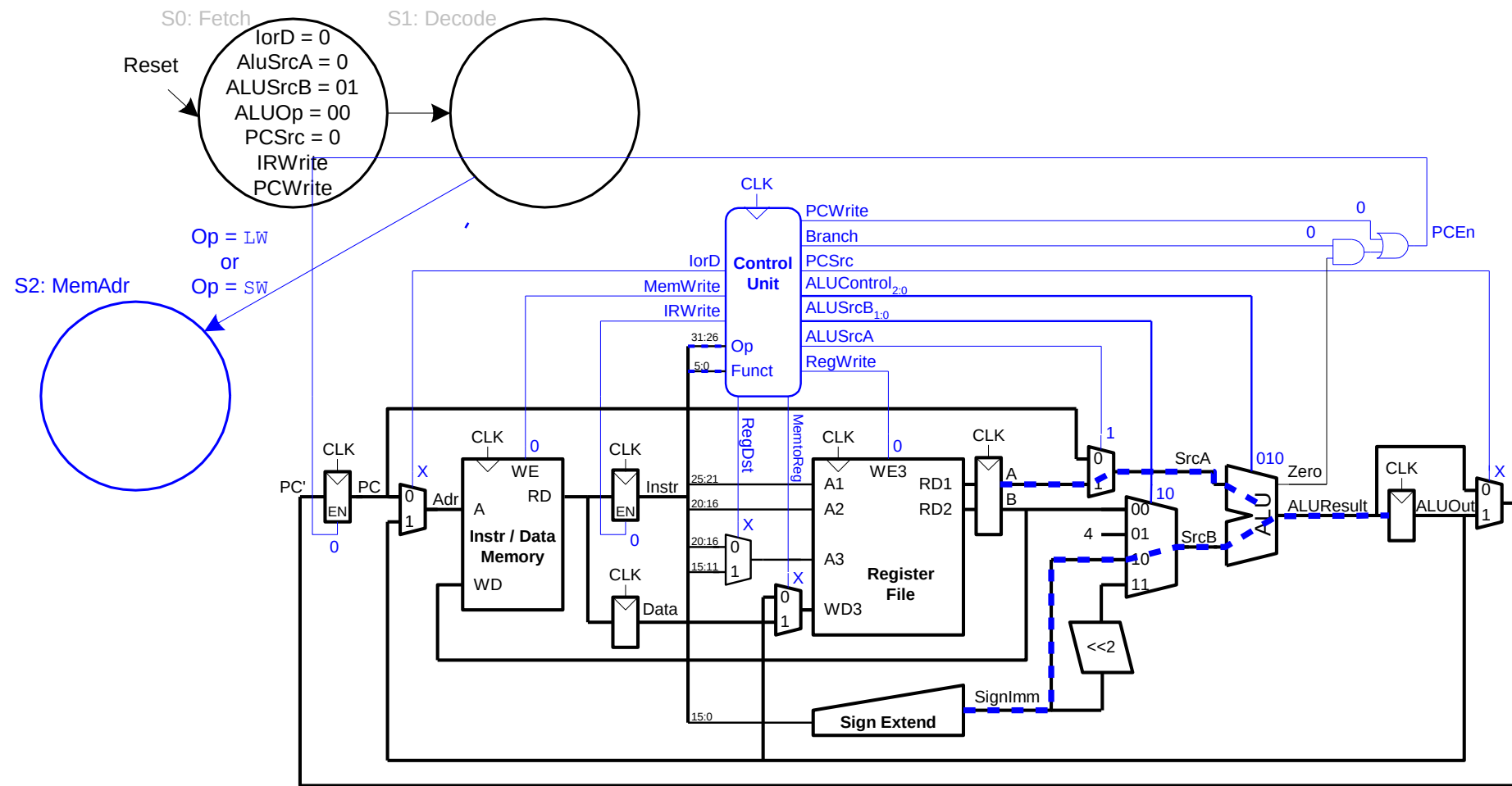
MAIN CONTROLLER FSM: FETCH



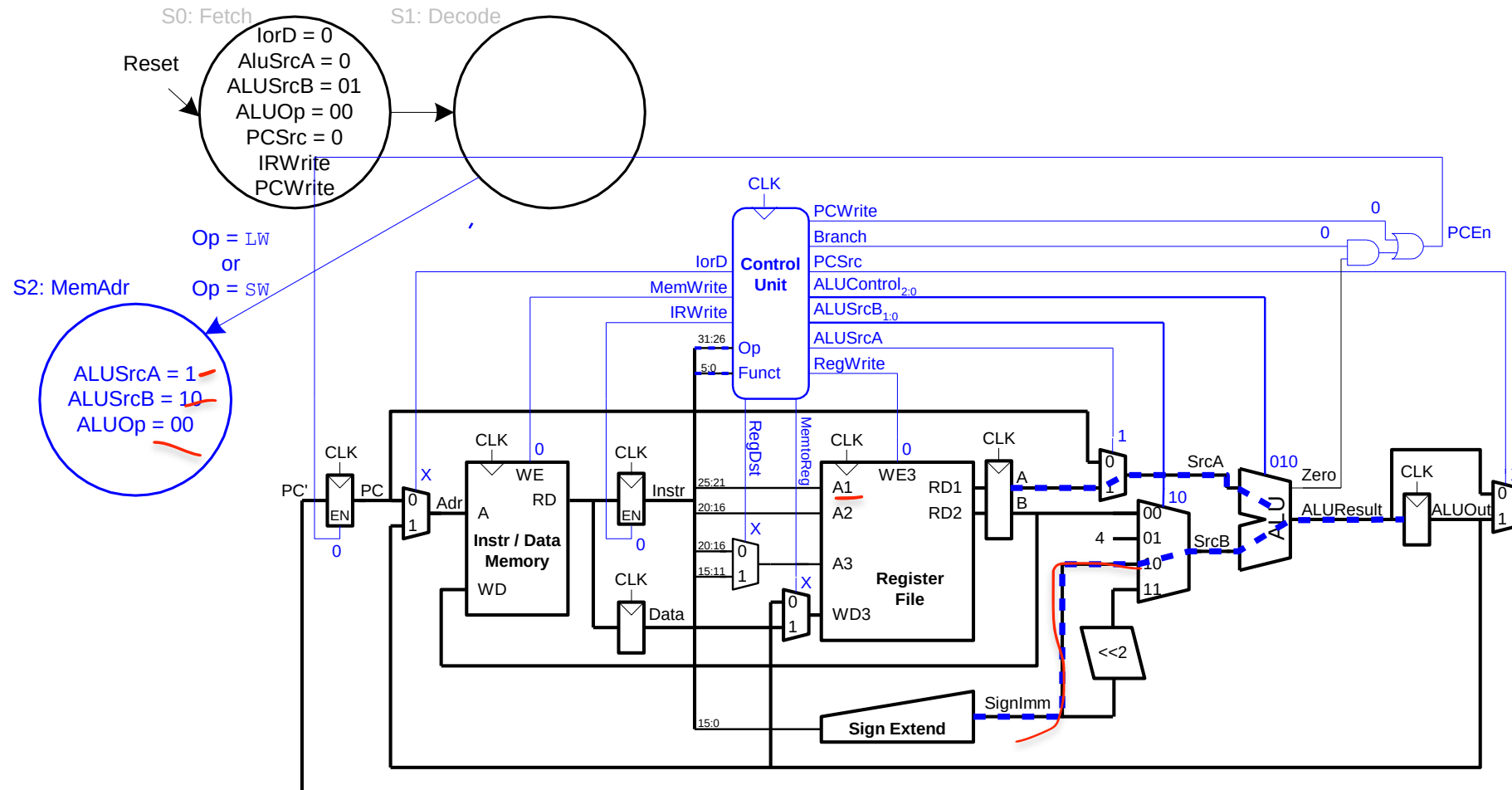
MAIN CONTROLLER FSM: DECODE



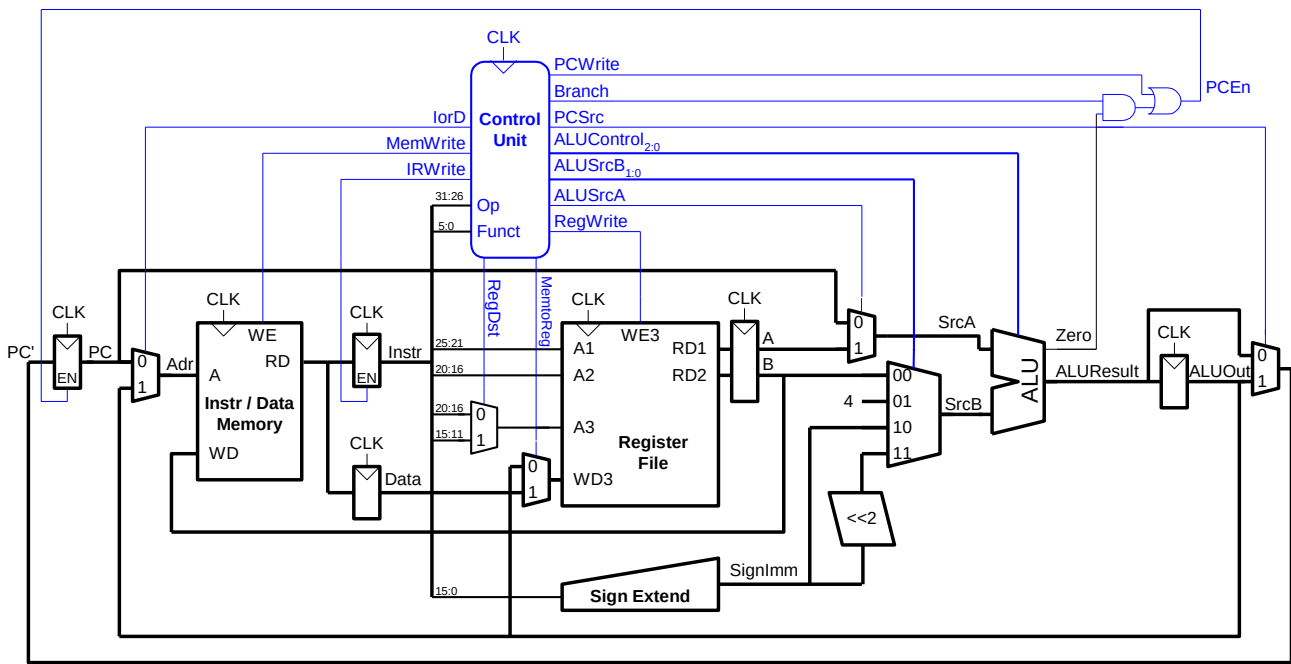
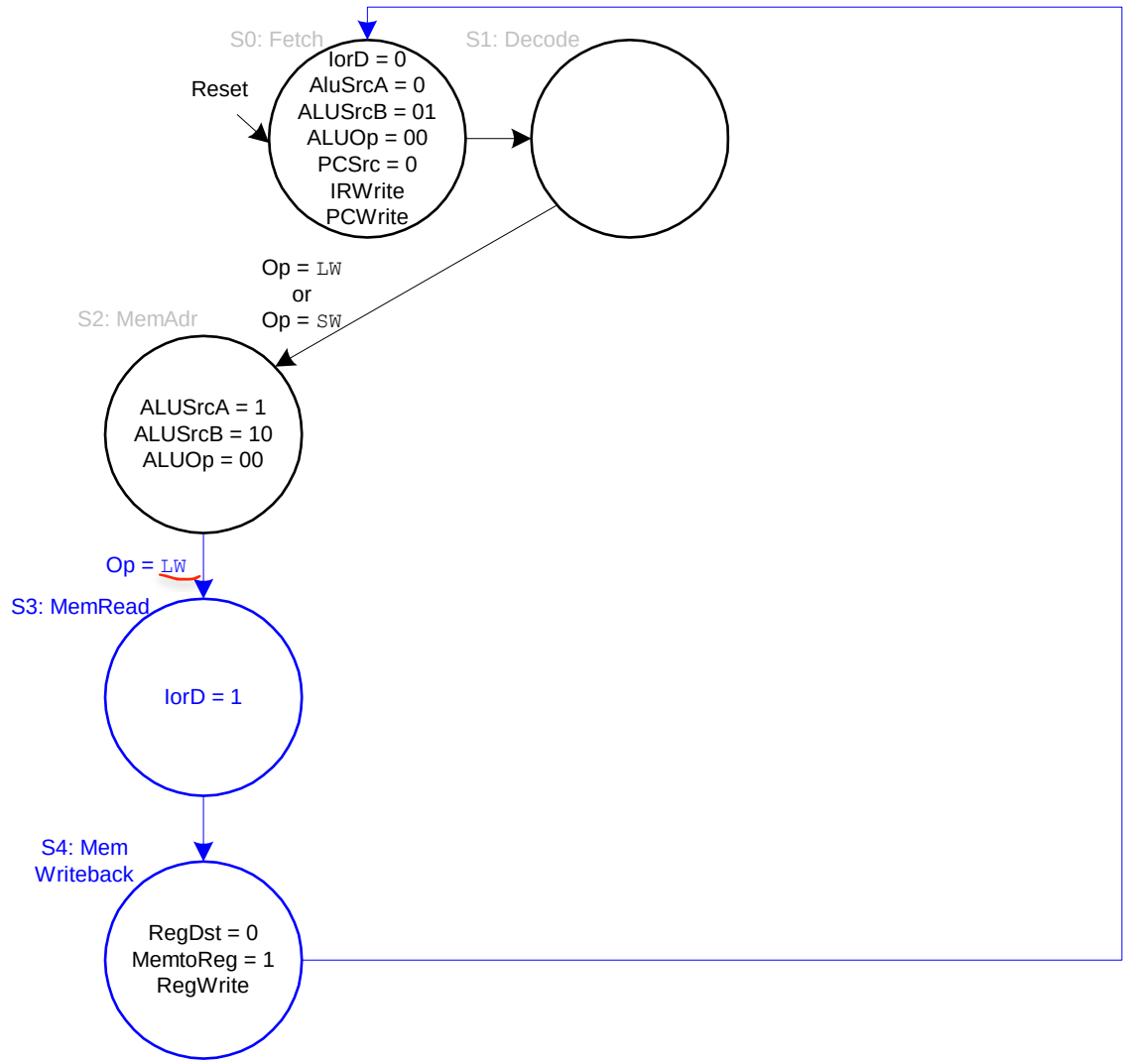
MAIN CONTROLLER FSM: ADDRESS CALCULATION



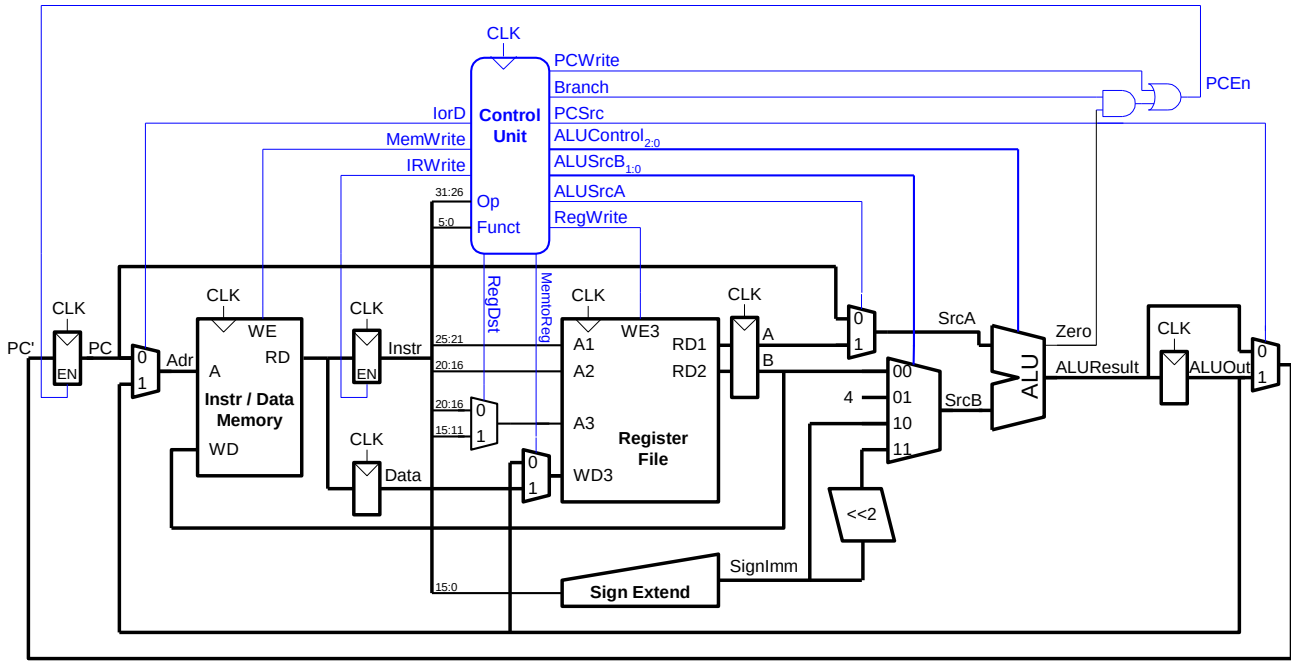
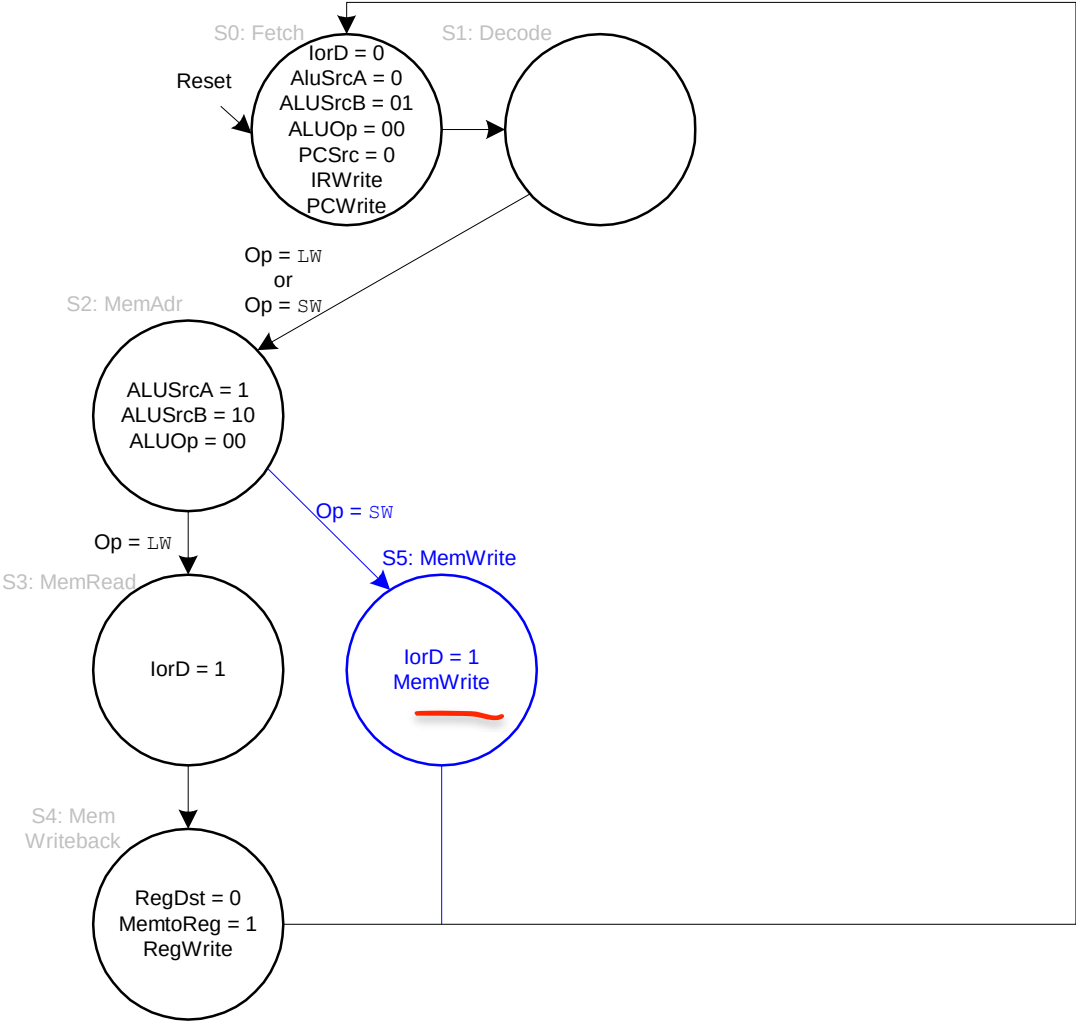
MAIN CONTROLLER FSM: ADDRESS CALCULATION



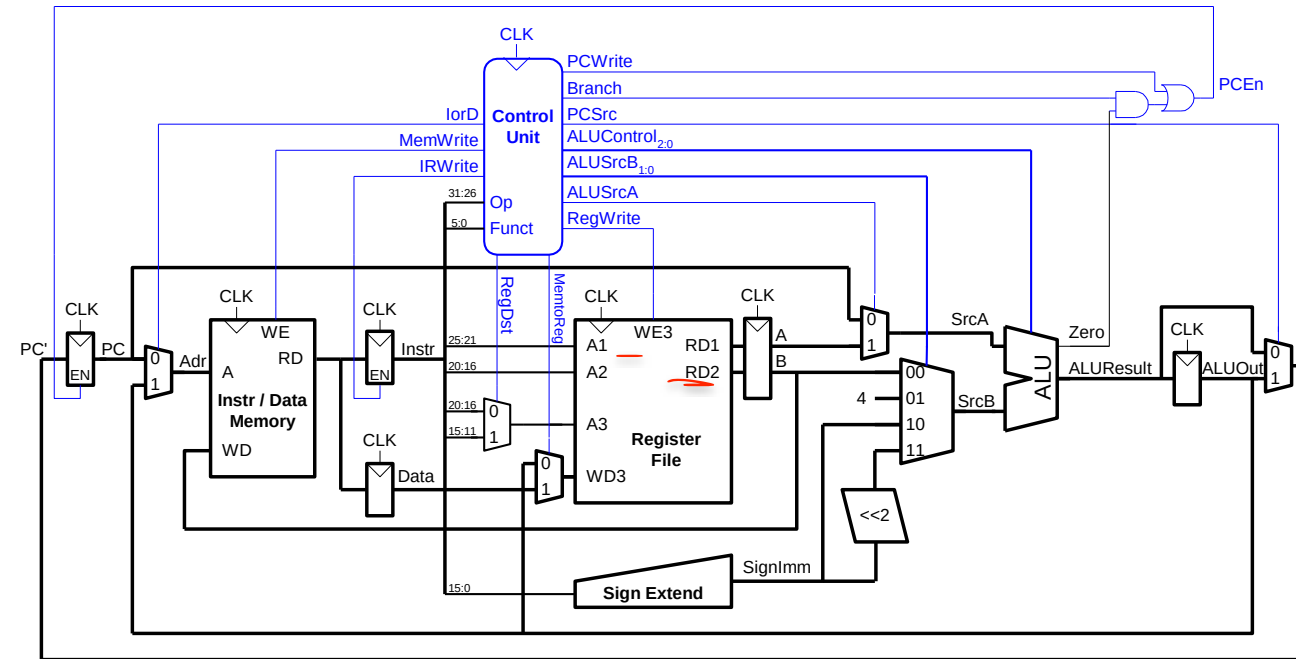
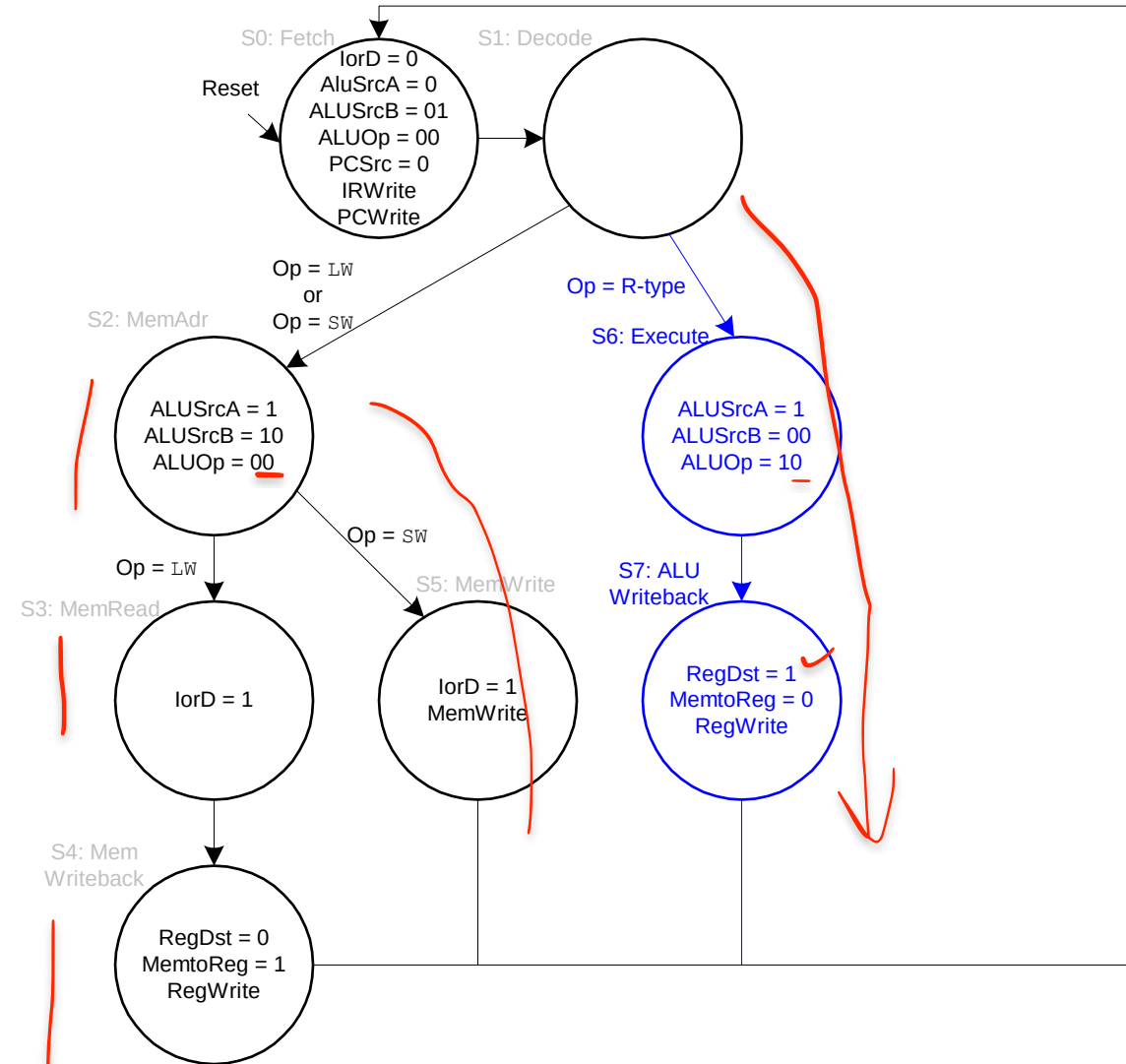
MAIN CONTROLLER FSM: LW



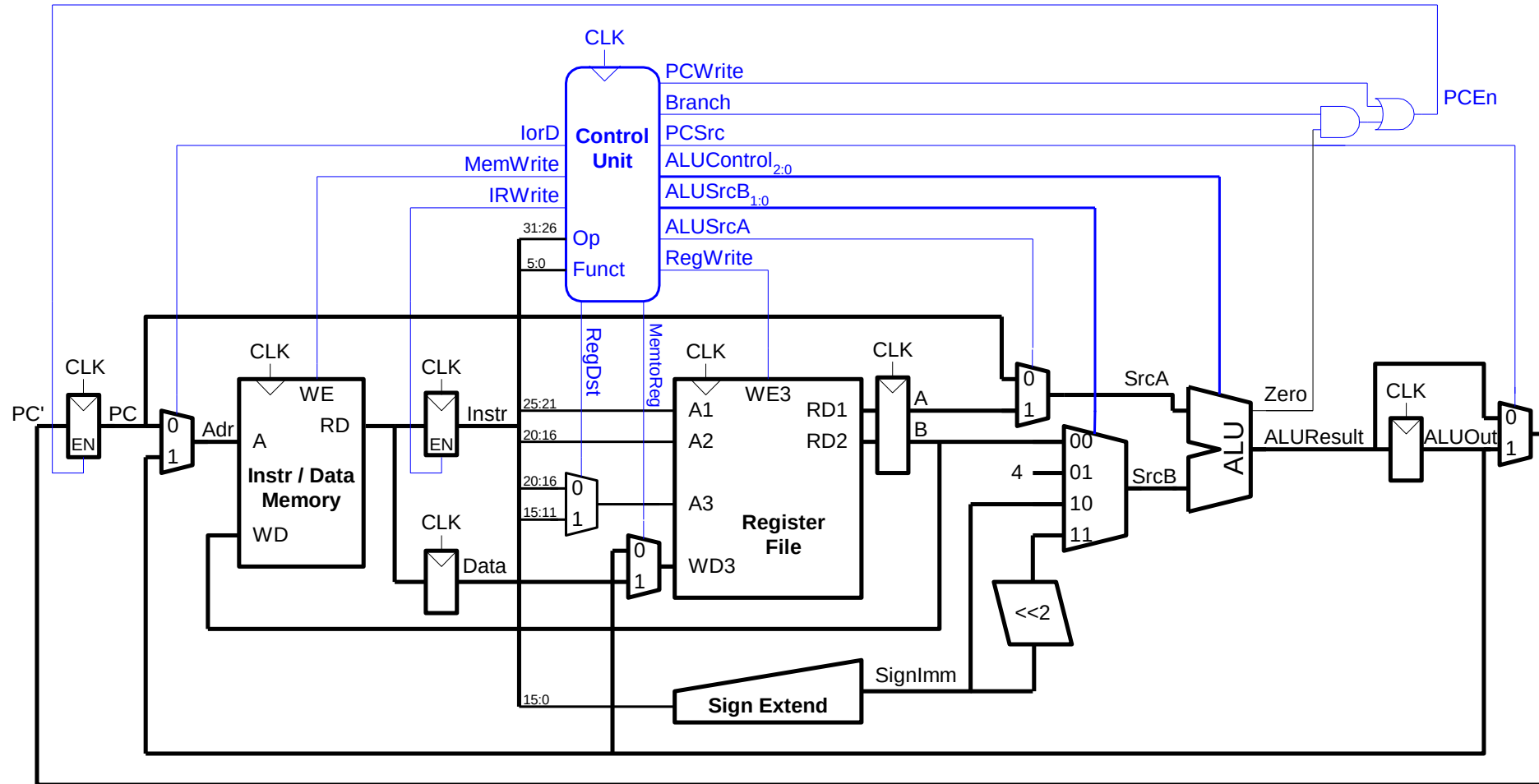
MAIN CONTROLLER FSM: SW



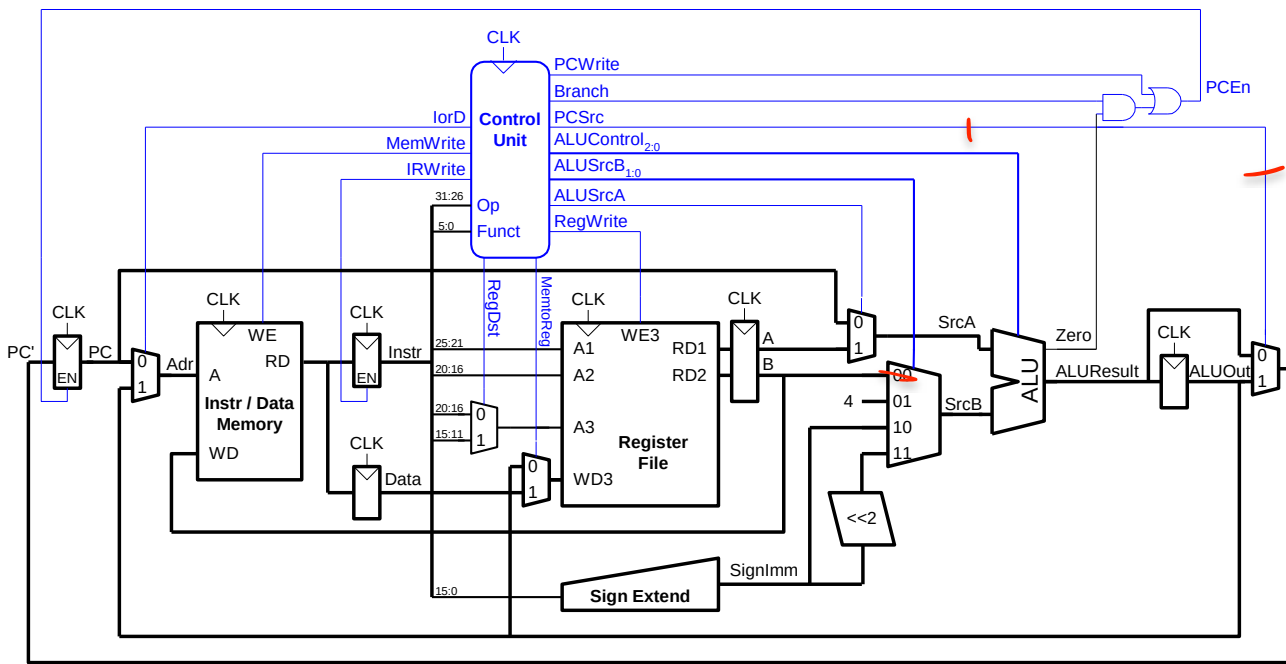
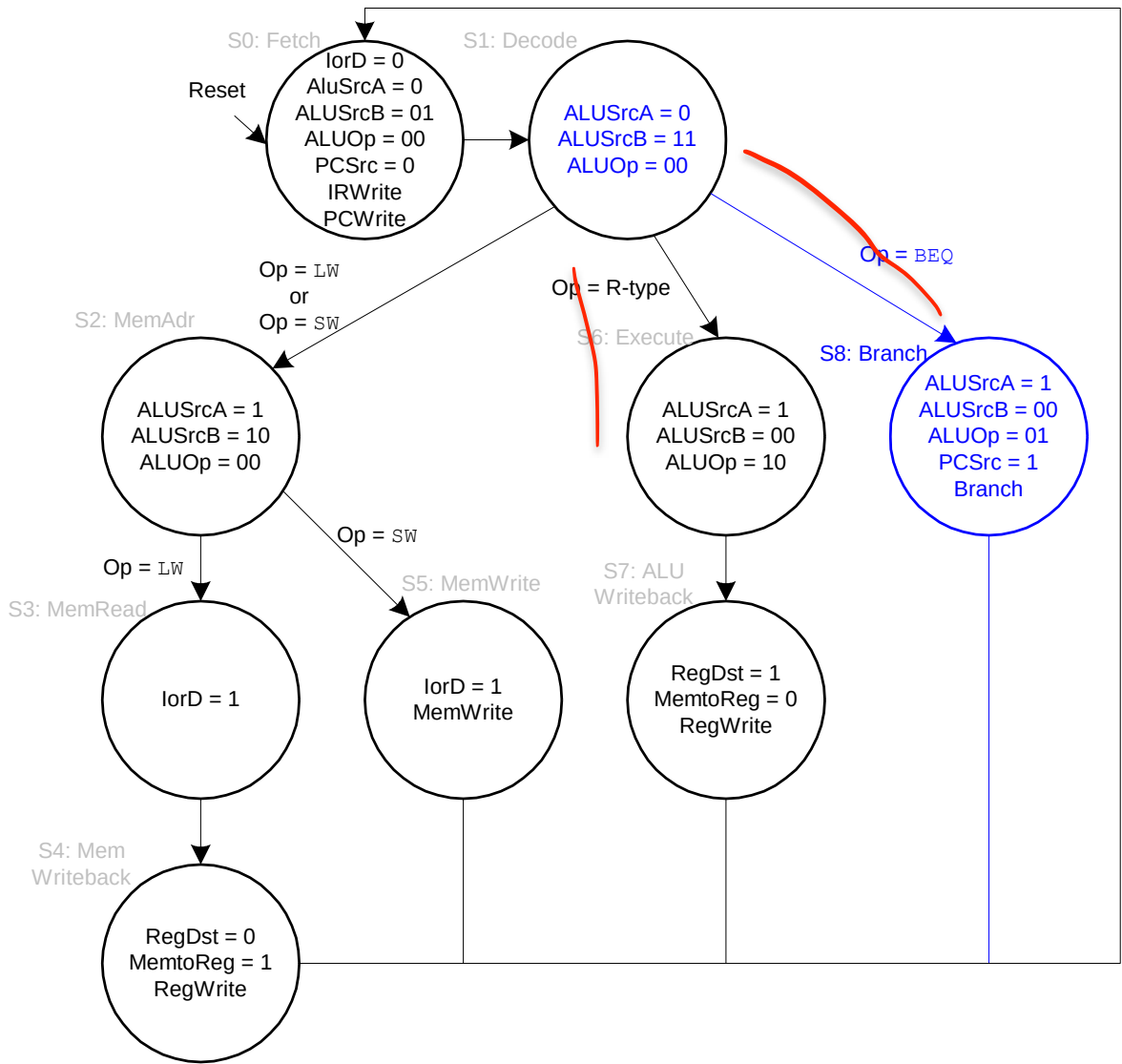
MAIN CONTROLLER FSM: R-TYPE



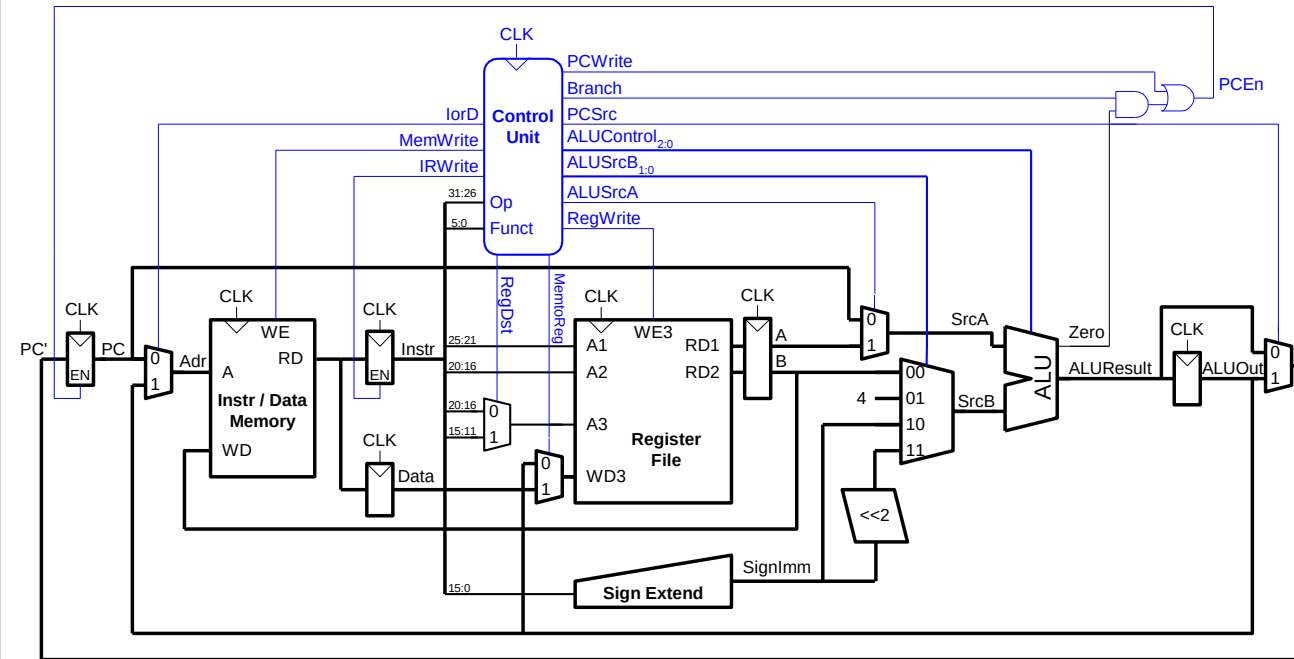
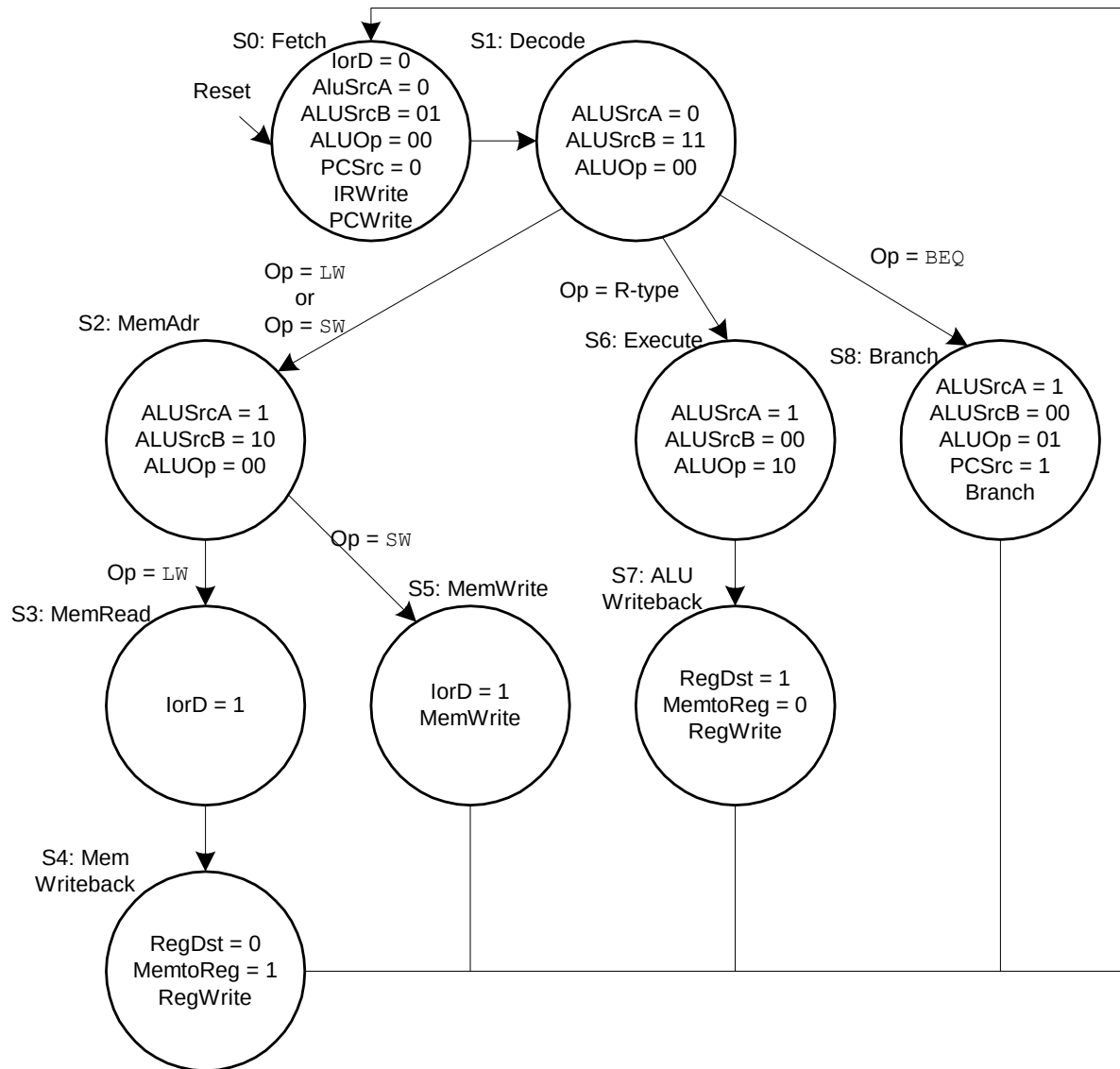
MAIN CONTROLLER FSM: R-TYPE



MAIN CONTROLLER FSM: BEQ



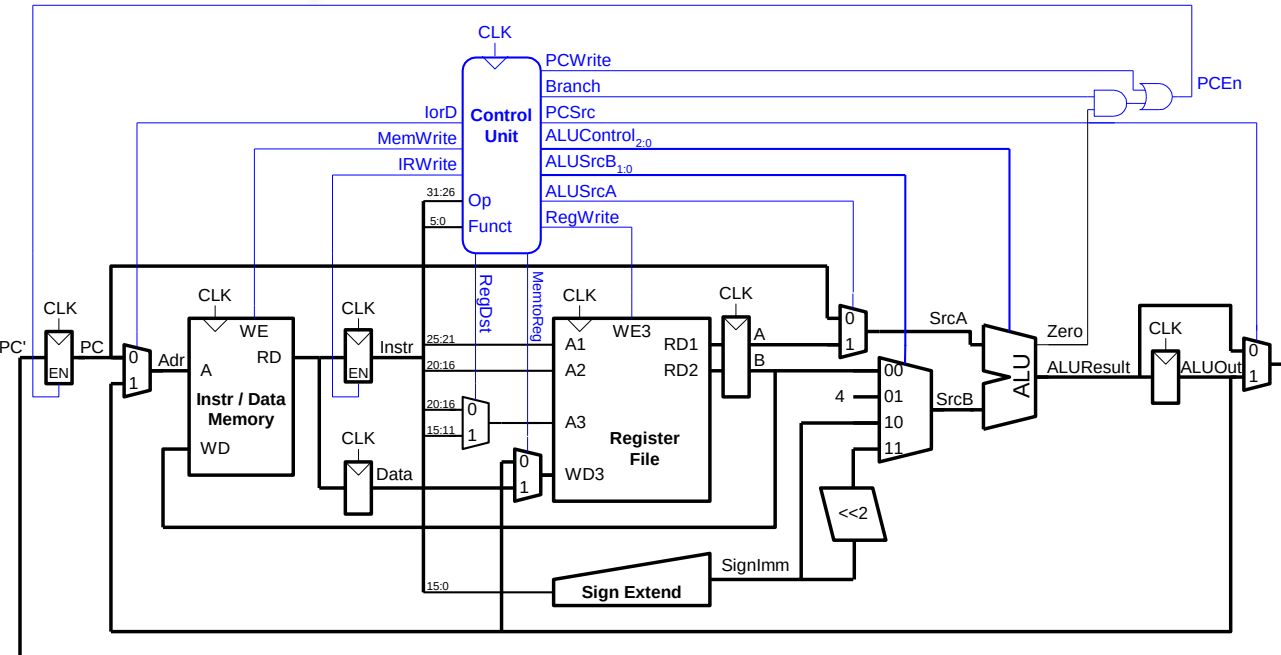
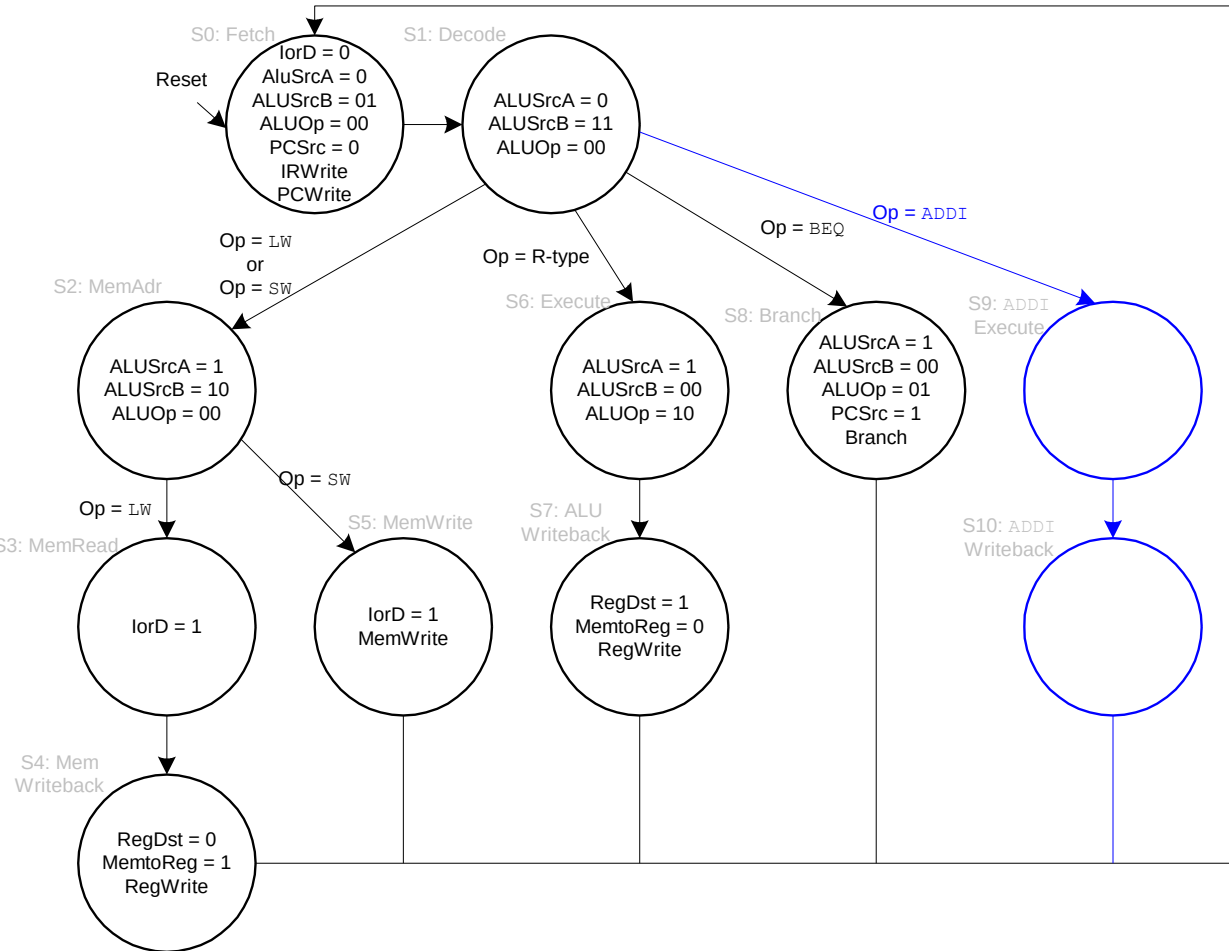
COMPLETE MULTICYCLE CONTROLLER FSM



MAIN CONTROLLER FSM: ADDI

ALUSrcA = X ✓
ALUSrcB = YY
ALUOp = 0

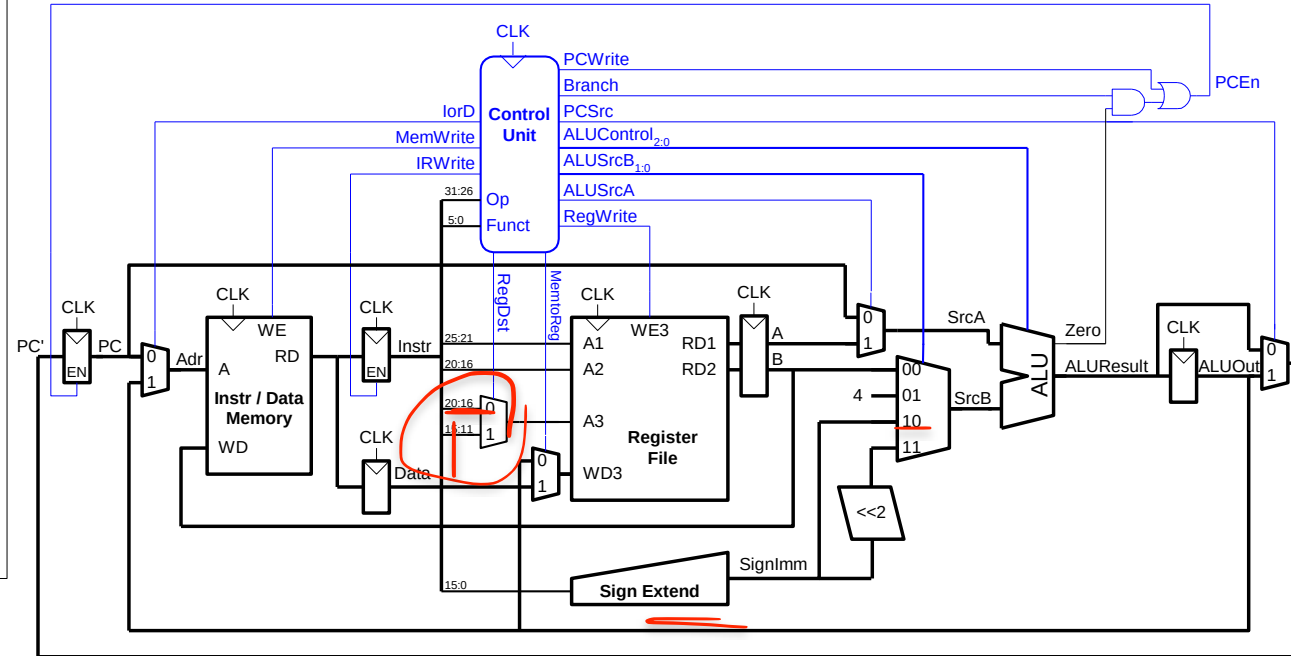
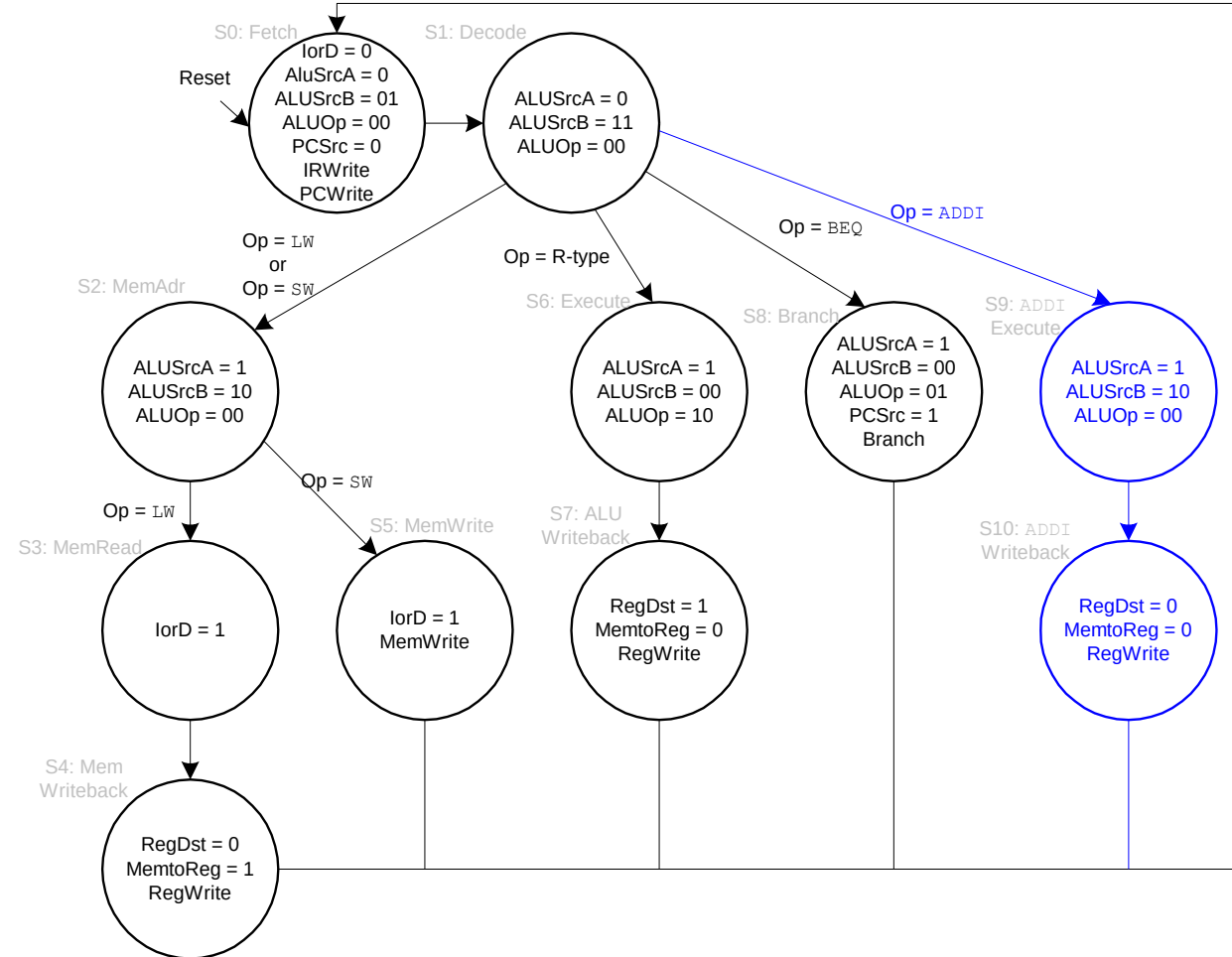
RegDST = X'
MemtoReg = Y'



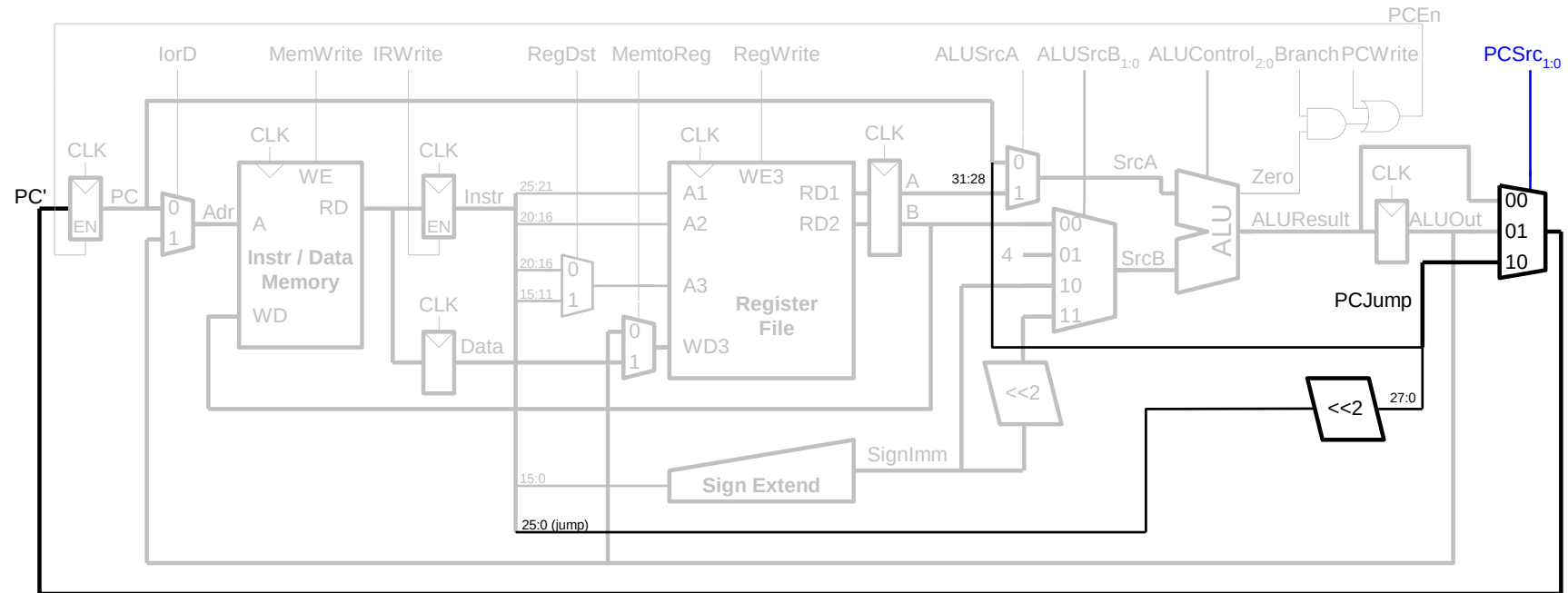
MAIN CONTROLLER FSM: ADDI

ALUSrcA = X
ALUSrcB = YY
ALUOp = 0

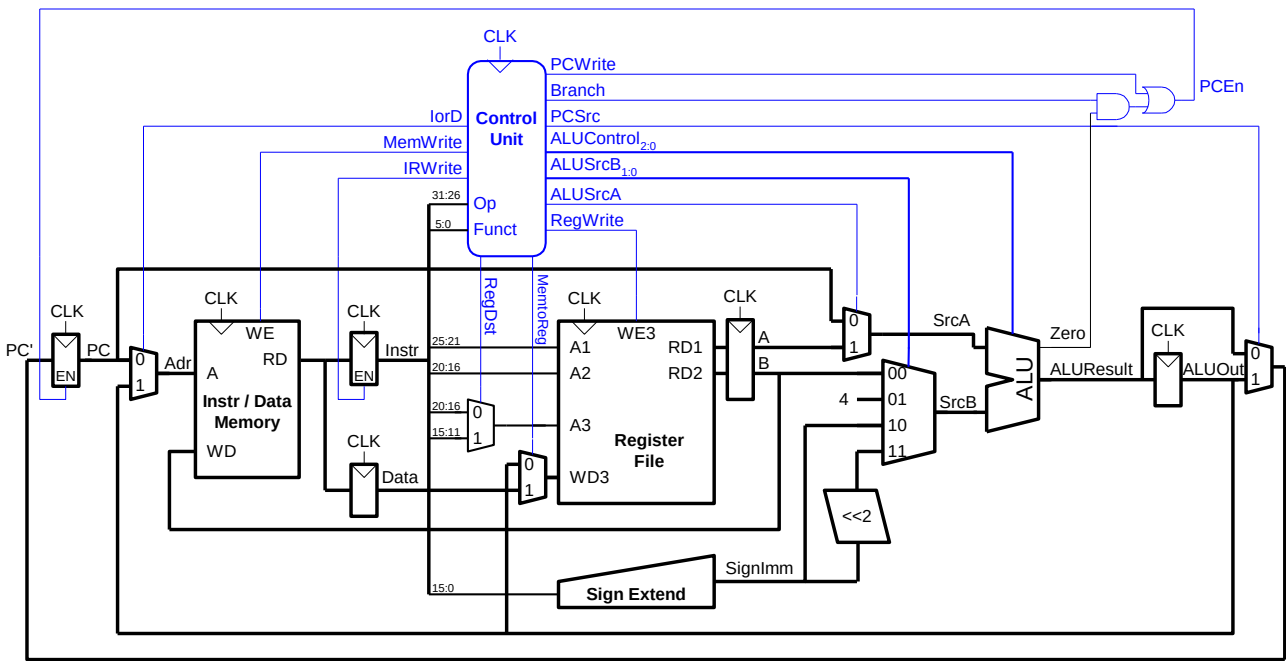
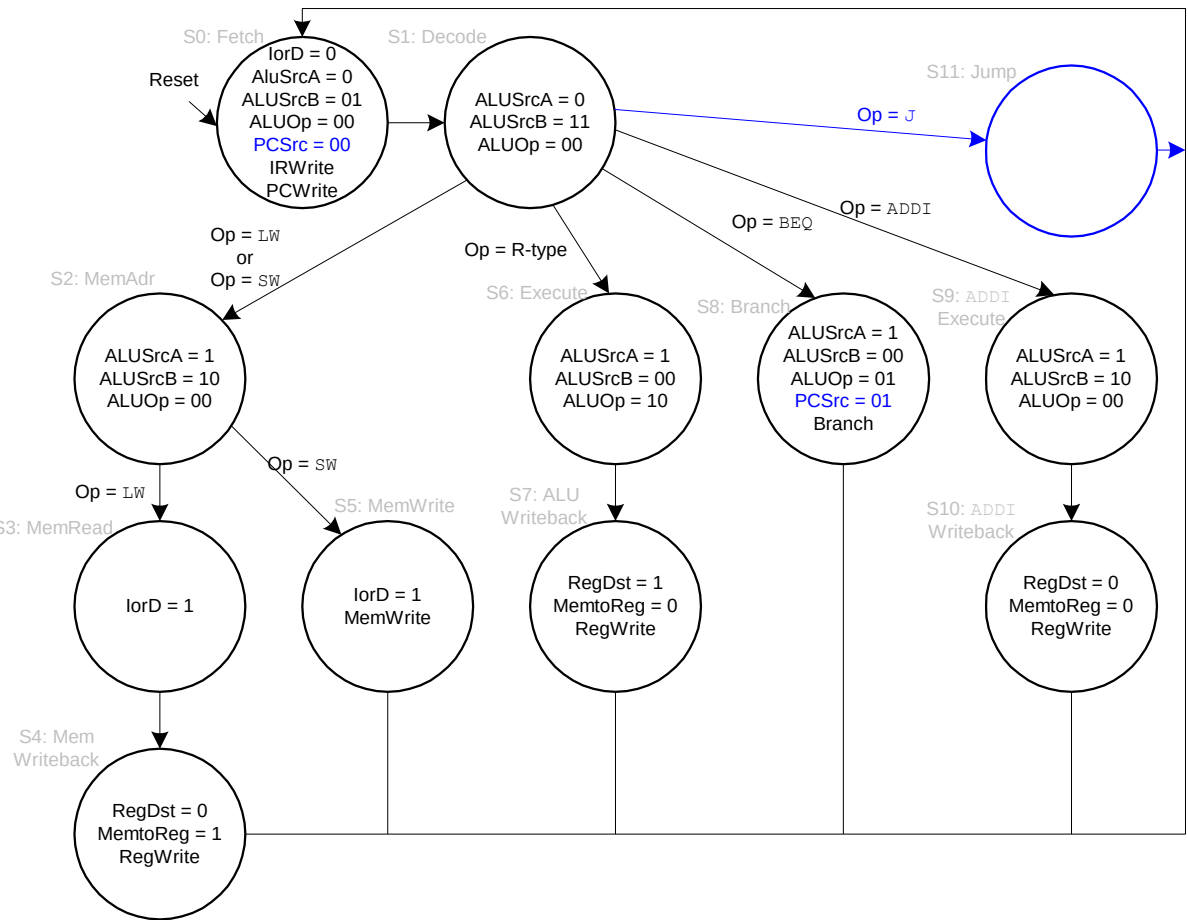
RegDST = X'
MemtoReg = Y'



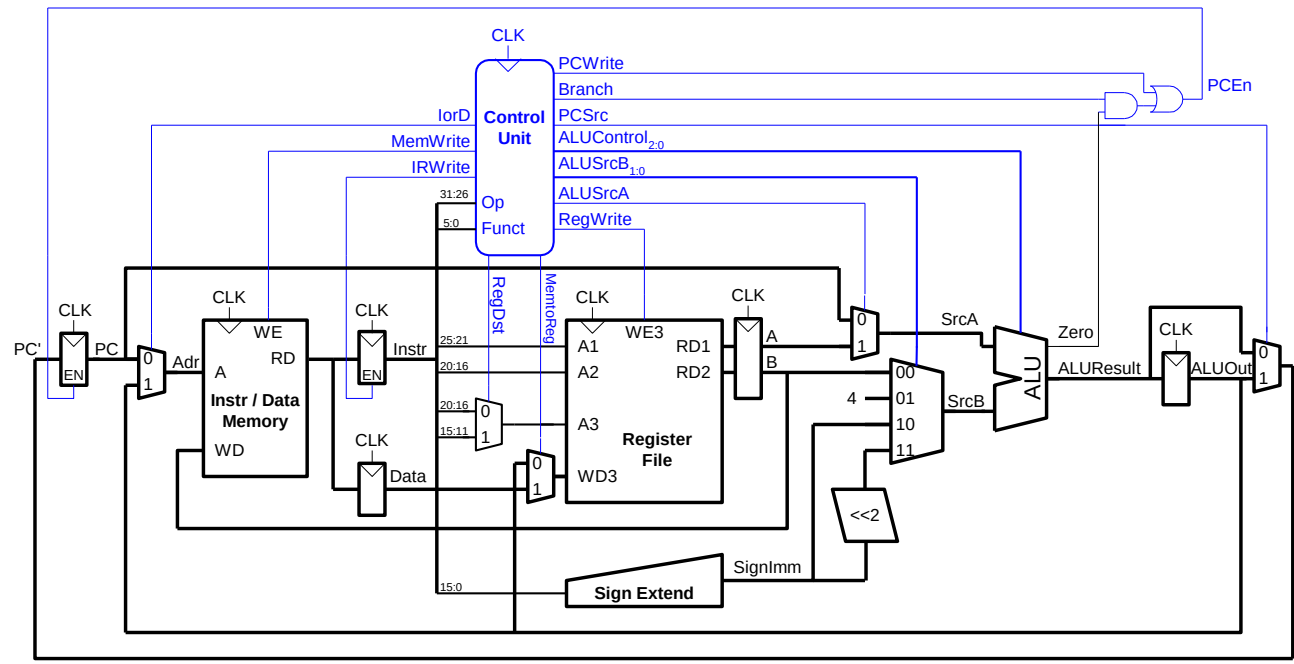
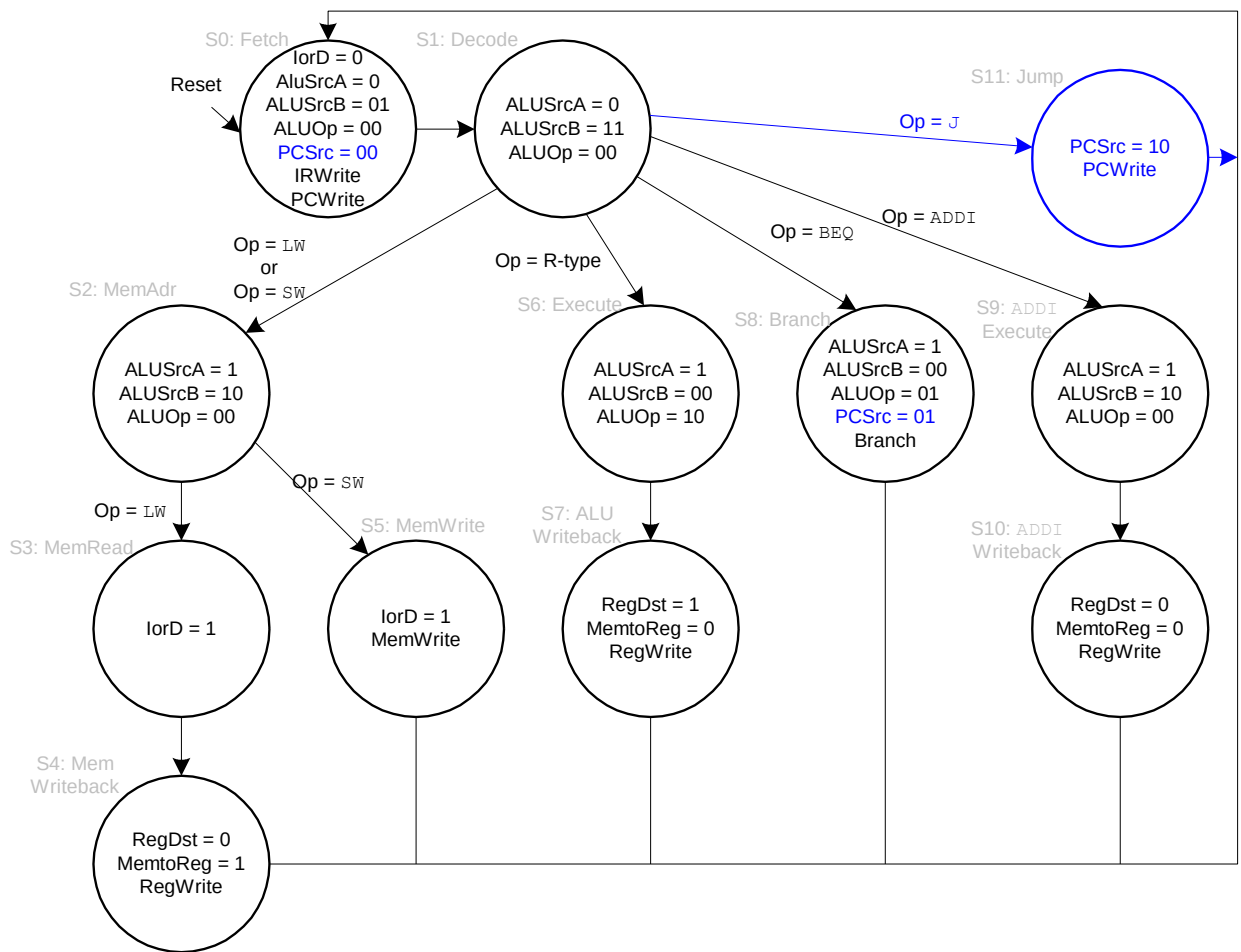
EXTENDED FUNCTIONALITY: J



CONTROL FSM: J



CONTROL FSM: J



MULTICYCLE PERFORMANCE

- Instructions take different number of cycles:

- 3 cycles: beq, j
- 4 cycles: R-Type, sw, addi
- 5 cycles: lw

- CPI is weighted average

- SPECINT2000 benchmark:

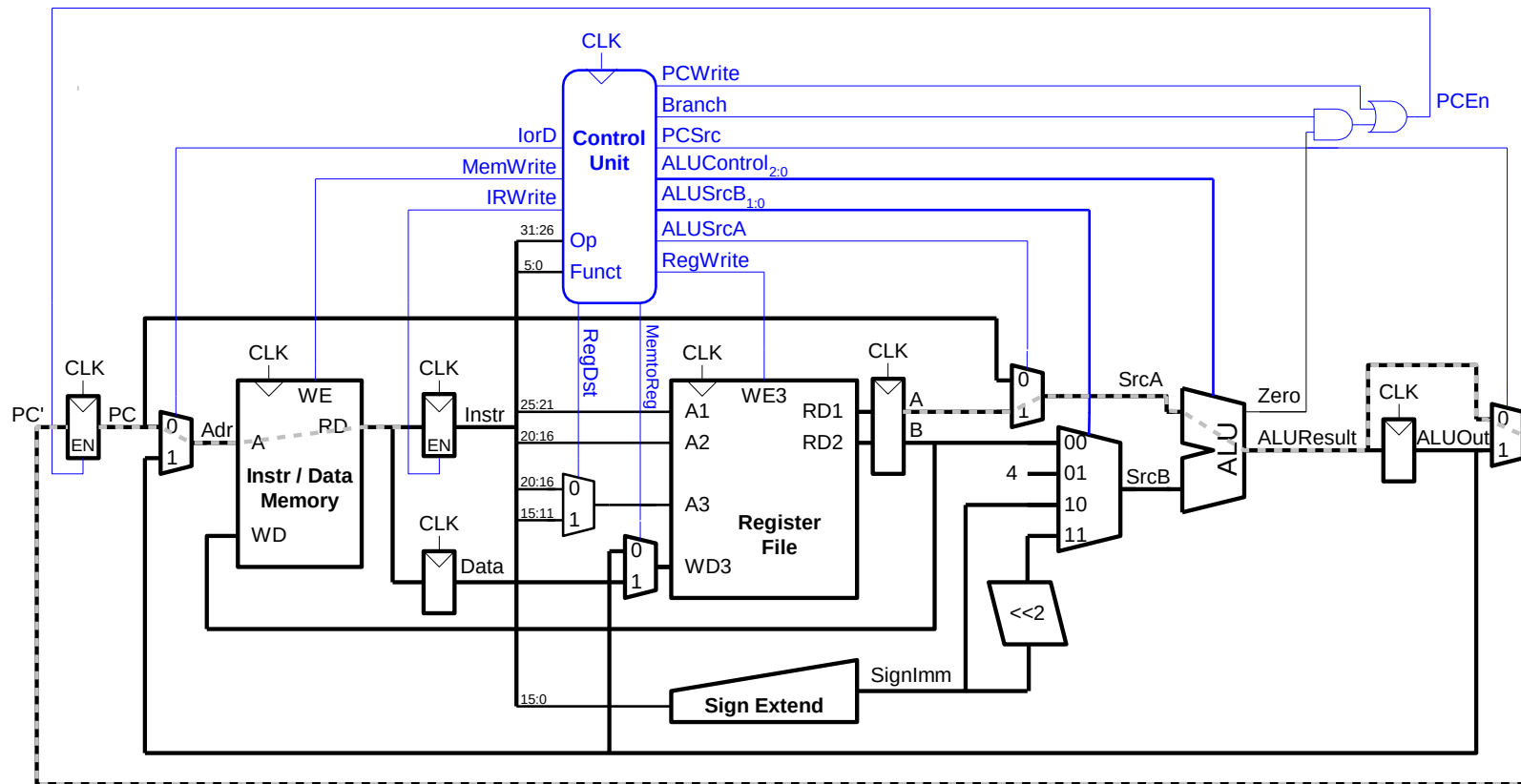
- 25% loads
- 10% stores
- 11% branches
- 2% jumps
- 52% R-type

$$\text{Average CPI} = \underline{(0.11 + 0.2)}(3) + \underline{(0.52 + 0.10)}(4) + \underline{(0.25)}(5) = 4.12$$

MULTICYCLE PERFORMANCE

- Multicycle critical path:

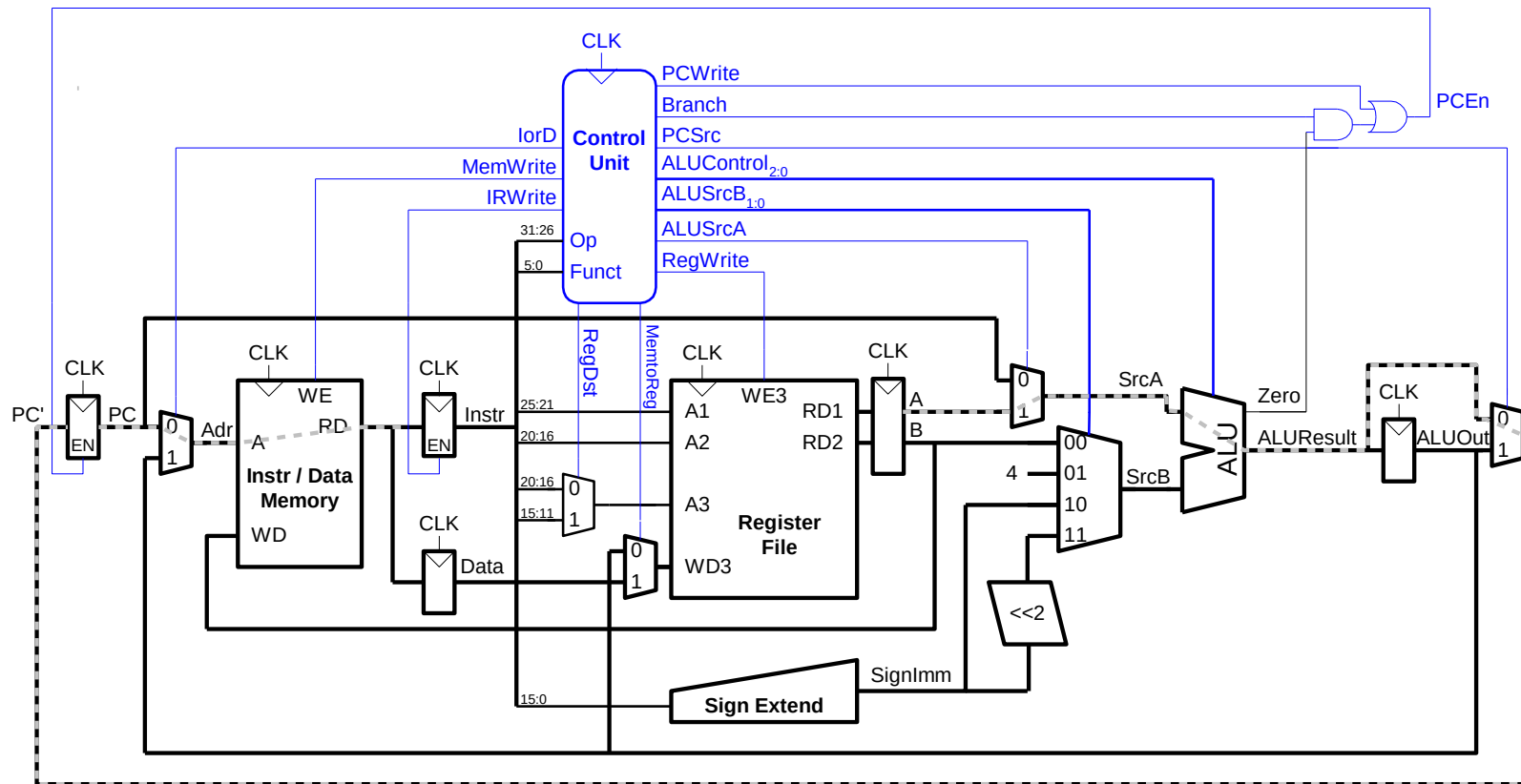
$$T_c =$$



MULTICYCLE PERFORMANCE

- Multicycle critical path:

$$T_c = \underline{t_{pcq}} + \underline{t_{mux}} + \underline{\max(t_{ALU} + t_{mux}, t_{mem})} + \underline{t_{setup}}$$



MULTICYCLE PERFORMANCE EXAMPLE

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq}	30 ✓
Register setup	t_{setup}	20 -
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$T_c =$$

MULTICYCLE PERFORMANCE EXAMPLE

Element	Parameter	Delay (ps)
Register clock-to-Q	t_{pcq_PC}	30
Register setup	t_{setup}	20
Multiplexer	t_{mux}	25
ALU	t_{ALU}	200
Memory read	t_{mem}	250
Register file read	t_{RFread}	150
Register file setup	$t_{RFsetup}$	20

$$\begin{aligned}T_c &= t_{pcq_PC} + t_{mux} + \max(t_{ALU} + t_{mux}, t_{mem}) + t_{setup} \\&= t_{pcq_PC} + t_{mux} + t_{mem} + t_{setup} \\&= [30 + 25 + 250 + 20] \text{ ps} \\&= 325 \text{ ps}\end{aligned}$$

MULTICYCLE PERFORMANCE EXAMPLE

- For a program with 100 billion instructions executing on a multicycle MIPS processor
 - $\text{CPI} = 4.12$ ✓
 - $T_c = 325 \text{ ps}$

Execution Time =

MULTICYCLE PERFORMANCE EXAMPLE

- For a program with 100 billion instructions executing on a multicycle MIPS processor
 - $\text{CPI} = 4.12$
 - $T_c = 325 \text{ ps}$

$$\begin{aligned}\text{Execution Time} &= (\# \text{ instructions}) \times \text{CPI} \times T_c \\ &= (100 \times 10^9)(4.12)(325 \times 10^{-12}) \\ &= \underline{133.9} \text{ seconds}\end{aligned}$$

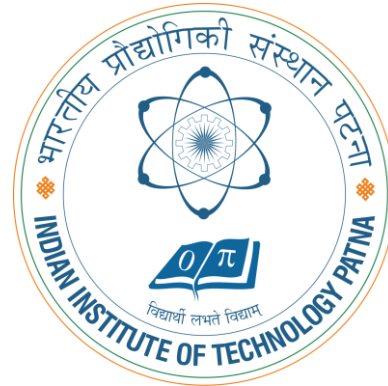
- This is **slower** than the single-cycle processor (92.5 seconds). Why?

MULTICYCLE PERFORMANCE EXAMPLE

- For a program with 100 billion instructions executing on a multicycle MIPS processor
 - $\text{CPI} = 4.12$
 - $T_c = 325 \text{ ps}$

$$\begin{aligned}\text{Execution Time} &= (\# \text{ instructions}) \times \text{CPI} \times T_c \\ &= (100 \times 10^9)(4.12)(325 \times 10^{-12}) \\ &= 133.9 \text{ seconds}\end{aligned}$$

- This is **slower** than the single-cycle processor (92.5 seconds). Why?
 - Not all steps the same length
 - Sequencing overhead for each step ($t_{pcq} + t_{\text{setup}} = 50 \text{ ps}$)



THANK YOU!